

Universidad
Tecnológica
de Pereira

Estabilidad de las explicaciones de redes neuronales de grafos para la detección de lavado de dinero bajo desbalance extremo de clases

Alejandro Gómez Huertas
Juan Diego Garzón Ovalle

Universidad Tecnológica de Pereira
Facultad de Ingenierías
Maestría en Ingeniería de Sistemas y Computación
Pereira, Colombia
2026

Estabilidad de las explicaciones de redes neuronales de grafos para la detección de lavado de dinero bajo desbalance extremo de clases

Alejandro Gómez Huertas
Juan Diego Garzón Ovalle

Tesis presentada como requisito parcial para optar al título de:

Magíster en Ingeniería de Sistemas y Computación

Director
Ph.D. Cristian Rosero Arias

Universidad Tecnológica de Pereira
Facultad de Ingenierías
Maestría en Ingeniería de Sistemas y Computación
Pereira, Colombia
2026

Resumen

[Por redactar al cerrar los Capítulos 5 a 7.]

Palabras clave: redes neuronales de grafos, explicabilidad, estabilidad de las explicaciones, lavado de dinero, desbalance de clases, Elliptic.

Abstract

[To be written once Chapters 5 to 7 are closed.]

Keywords: graph neural networks, explainability, explanation stability, money laundering, class imbalance, Elliptic.

Índice general

Resumen **i**

Abstract **ii**

Índice de figuras **vii**

Índice de tablas **ix**

Lista de acrónimos **x**

1. Introducción **1**

1.1. Planteamiento del problema 1

1.2. Una limitación del conjunto de datos que condiciona el diseño 2

1.3. Preguntas de investigación 2

1.4. Hipótesis 3

1.5. Objetivos 3

1.5.1. Objetivo general 3

1.5.2. Objetivos específicos 3

1.6. Contribuciones 4

1.7. Alcance y limitaciones 4

1.8. Estructura del documento 5

2. Marco teórico **6**

2.1. Lavado de dinero en criptomonedas 6

2.2. Redes neuronales de grafos 6

2.3. Desbalance de clases 7
2.4. Explicabilidad de modelos 8
2.5. Evaluación de las explicaciones 9

3. Estado del arte **11**
3.1. Detección de lavado de dinero con grafos 11
3.2. Explicabilidad en la detección de lavado de dinero 12
3.3. Estabilidad de las explicaciones y desbalance de clases 12
3.4. Vacío de investigación 13

4. Metodología **14**
4.1. Datos 14
4.2. Partición temporal y exclusión del cierre del mercado 15
4.3. Escenarios de desbalance 16
4.4. Modelos y entrenamiento 17
4.5. Explicadores 18
4.6. Medidas de estabilidad 19
4.7. Control de pesos aleatorios 20
4.8. Análisis estadístico 20

5. Resultados **22**
5.1. La partición sin el cierre recupera la prueba 22
5.2. Rendimiento según el escenario y la pérdida 23
5.3. H1: el nivel de desbalance no cambia la estabilidad 25
5.4. H2: no hay un orden estable entre arquitecturas 27
5.5. H3: la técnica de compensación no cambia la estabilidad 28
5.6. Qué parte de la estabilidad depende del modelo 29
5.7. Síntesis 30

6. Discusión **32**

7. Conclusiones **33**

Referencias **34**

A. Corrida con la partición original **39**

A.1. La prueba cae al nivel del azar 39

A.2. Lo que se mantiene en las dos particiones 39

A.3. Lo que cambia 40

B. Eje sintético **41**

B.1. Diseño 41

B.2. Resultados 41

B.3. Advertencias 42

C. Reproducibilidad **43**

C.1. Entorno 43

C.2. Corrida principal (sin el cierre del mercado) 43

C.3. Tablas, figuras y bibliografía del manuscrito 43

C.4. Determinismo 44

Índice de figuras

4.1. (a) Transacciones ilícitas (barras) y lícitas (línea) por paso de tiempo en Elliptic. La línea discontinua marca el cierre del mercado ilegal en el paso 43. (b) Partición original y partición sin el cierre usada en el estudio principal; en esta última los pasos 43 a 49 no se usan. 16

5.1. Rendimiento en prueba con la partición original (prueba en los pasos 43 a 49, después del cierre) y con la partición sin el cierre (prueba en los pasos 36 a 42). Escenario nativo, pérdida elegida por validación. Las barras son la media de tres semillas, los puntos cada semilla y las líneas discontinuas el valor del azar: la prevalencia en (a) y 0,5 en (b). 23

5.2. Curvas de precisión y exhaustividad (a) y ROC (b) en la prueba de la partición sin el cierre, escenario nativo, con la pérdida elegida por validación para cada arquitectura. Trazo grueso: semilla 42; trazos finos: semillas 43 y 44. Entre paréntesis, el área media de las tres semillas. 24

5.3. PR-AUC de validación por escenario y pérdida (media y desviación estándar de tres semillas). La línea discontinua es el valor del azar (0,037). 24

5.4. Estabilidad entre réplicas por escenario y explicador, con las tres medidas. Pesos por clase; media de 9 modelos (3 arquitecturas que aprenden por 3 semillas) e intervalo *bootstrap* del 95 %. 26

5.5. Estabilidad de GNNExplainer por arquitectura con las tres medidas (pesos por clase, media de los cuatro escenarios principales). Cada punto es una semilla; la línea horizontal es la media. 27

5.6. Control de pesos aleatorios en el escenario nativo con pesos por clase (12 modelos: 4 arquitecturas por 3 semillas). Para cada explicador, la estabilidad entre réplicas con la red entrenada y con una red de pesos aleatorios, y el acuerdo entre la explicación de la red entrenada y la de la red aleatoria. 29

5.7. Acuerdo entre explicadores sobre el mismo modelo (GraphSAGE, GAT y TAGCN; escenarios principales; todas las pérdidas y semillas), con el Spearman global (a) y el Jaccard top-10 (b). 31

Índice de tablas

4.1. Factores del diseño experimental. Cada combinación se entrena con tres semillas de modelo, lo que da $4 \times 6 \times 3 \times 3 = 216$ modelos. 14

4.2. Transacciones por parte de cada partición. Negativas son lícitas más sin etiqueta; la razón es negativas por cada ilícita. 16

5.1. Rendimiento en el escenario nativo con la pérdida elegida por validación (media $\pm$ desviación estándar de tres semillas). Filtro: modelos que superan el filtro de calidad en validación. 23

5.2. PR-AUC de validación por escenario, arquitectura y pérdida (media de tres semillas). 25

5.3. Modelos que superan el filtro de calidad en validación (F1 $\geq$ 0,30 y MCC $\geq$ 0,15 con umbral 0,5), de 18 por celda (6 escenarios por 3 semillas). 25

5.4. Estabilidad entre réplicas por escenario (pesos por clase, 9 modelos). *p* Friedman: igualdad de los seis escenarios. *p* TOST: equivalencia entre nativo y 1:200 con margen 0,05 (un valor bajo indica equivalencia). 26

5.5. Estabilidad por arquitectura (pesos por clase, media de los escenarios principales, 3 semillas). *p* (4): Friedman con las cuatro arquitecturas; *p* (3): sin GCN. Primera: número de semillas en que cada arquitectura obtiene el mayor valor. . 27

5.6. Estabilidad por técnica de compensación (3 arquitecturas que aprenden, media de los escenarios principales). PC $>$ FL y PC $>$ SA: en cuántos de los 9 pares de arquitectura y semilla pesos por clase supera a *focal loss* o a sin ajuste, con el *p* de la prueba de signos. 28

5.7. Estabilidad de PGExplainer sobre las aristas del subgrafo: Spearman entre réplicas y Jaccard de las 20 aristas principales (escenarios principales, media de 3 semillas). 28

5.8. Control de pesos aleatorios (media de 12 modelos). 29

5.9. Veredicto de las hipótesis con la partición sin el cierre. 31

B.1. Eje sintético: estabilidad (Spearman global), plausibilidad de variables y fidelidad (fid+), con su valor al azar entre paréntesis. Medias sobre 36 configuraciones por fila. Fuente: `results_phase1_v4/summary_v4.csv`. 42

Lista de acrónimos

AML	*Anti-Money Laundering*, prevención del lavado de dinero
EG	*Expected Gradients*
F1	Media armónica de precisión y exhaustividad
GAT	*Graph Attention Network*, red de grafos con atención
GCN	*Graph Convolutional Network*, red convolucional de grafos
GNN	*Graph Neural Network*, red neuronal de grafos
GraphSAGE	*Graph Sample and Aggregate*
IG	*Integrated Gradients*
MCC	Coeficiente de correlación de Matthews
PR-AUC	Área bajo la curva de precisión y exhaustividad
ROC-AUC	Área bajo la curva ROC
SMOTE	*Synthetic Minority Over-sampling Technique*
TAGCN	*Topology Adaptive Graph Convolutional Network*
TOST	*Two One-Sided Tests*, prueba de equivalencia
XAI	*Explainable Artificial Intelligence*, inteligencia artificial explicable

Capítulo 1

Introducción

1.1. Planteamiento del problema

La detección de lavado de dinero se apoya cada vez más en modelos de aprendizaje automático que examinan transacciones en busca de patrones anómalos [1, 2]. En el caso de las criptomonedas, las transacciones forman de manera natural un grafo, en el que cada transacción es un nodo y cada transferencia de fondos entre transacciones es una arista. Las redes neuronales de grafos (*Graph Neural Networks*, GNN) aprovechan esa estructura combinando los atributos de cada transacción con los de sus vecinas [3, 4], y se han convertido en una de las familias de modelos más estudiadas para la detección de fraude financiero [5].

En un entorno regulado no basta con que el modelo acierte. El analista que revisa una alerta necesita saber por qué la transacción fue marcada como sospechosa, y esa justificación debe poder sostenerse ante un auditor [6, 7]. Las GNN son modelos opacos, por lo que la justificación se obtiene con métodos de explicabilidad post hoc que asignan una importancia a cada variable o a cada arista que intervino en la predicción [8]. Esos métodos, sin embargo, son en su mayoría estocásticos u optimizan una máscara a partir de una inicialización aleatoria. Si se ejecutan dos veces sobre la misma transacción y el mismo modelo, pueden devolver explicaciones distintas. Una explicación que cambia cada vez que se pide no sirve como evidencia.

A esa fuente de variación se suma otra, propia del dominio. Las transacciones ilícitas son una fracción muy pequeña del total. En el conjunto de datos Elliptic, que es la referencia pública más usada para este problema [9], hay 4.545 transacciones etiquetadas como ilícitas frente a más de 199.000 lícitas o sin etiqueta. Bajo un desbalance así, los modelos aprenden la clase rara a partir de pocos ejemplos [10, 11], y es razonable esperar que sus explicaciones sobre esa clase también sean más frágiles. Las técnicas que compensan el desbalance, como reponderar la función de pérdida o generar ejemplos sintéticos de la clase rara [12, 13], modifican lo que el modelo aprende y podrían modificar también lo que el explicador reporta.

La literatura ha evaluado la calidad de los explicadores de GNN principalmente en grafos sintéticos o en tareas de química y redes sociales [14, 15], y la evaluación de la estabilidad de las explicaciones se ha estudiado sobre todo fuera de los grafos [16, 17]. No se encontró un estudio que mida, con un diseño factorial y réplicas independientes, cómo cambian la estabilidad de las explicaciones de distintas GNN cuando se varían el nivel de desbalance y la técnica que lo compensa, en un problema real de lavado de dinero. Ese es el vacío que aborda esta tesis.

1.2. Una limitación del conjunto de datos que condiciona el diseño

Elliptic cubre 49 pasos de tiempo separados aproximadamente dos semanas entre sí. En el paso 43 se cerró un mercado ilegal importante de la red, y los autores del conjunto de datos reportaron que, a partir de ese momento, ningún modelo era capaz de detectar las nuevas transacciones ilícitas [9]. La partición temporal habitual entrena con los pasos 1 a 34, valida con los pasos 35 a 42 y evalúa con los pasos 43 a 49, de modo que la prueba cae completa después del cierre. En la primera versión de los experimentos de esta tesis, todos los modelos quedaron en el nivel del azar en esa prueba (Anexo A).

Por esa razón, el estudio principal usa solo los pasos 1 a 42, anteriores al cierre, y los reparte en entrenamiento (1 a 29), validación (30 a 35) y prueba (36 a 42). Los pasos 43 a 49 no se usan. La decisión, sus costos y la comparación con la partición habitual se exponen en el Capítulo 4.

1.3. Preguntas de investigación

Pregunta principal. ¿Cómo cambia la estabilidad de las explicaciones que producen los métodos de explicabilidad sobre redes neuronales de grafos entrenadas para detectar transacciones ilícitas en Elliptic, cuando se varía el nivel de desbalance de clases, la arquitectura de la red y la técnica empleada para compensar el desbalance?

Preguntas específicas.

1. ¿Disminuye la estabilidad de las explicaciones a medida que el desbalance de clases con el que se entrena el modelo se hace más extremo?
2. ¿Producen las distintas arquitecturas (GCN, GraphSAGE, GAT y TAGCN) explicaciones con distinta estabilidad?
3. ¿Cambia la estabilidad de las explicaciones según la técnica usada para compensar el desbalance (sin ajuste, pesos por clase o *focal loss*)?

4. ¿Refleja la estabilidad medida lo que el modelo aprendió, o se debe principalmente a los datos de entrada y al propio explicador?

1.4. Hipótesis

Las hipótesis se formularon antes de observar los resultados y se contrastan en el Capítulo 5.

H1 (desbalance). La estabilidad de las explicaciones disminuye a medida que el desbalance de clases del entrenamiento se hace más extremo, porque el modelo dispone de menos ejemplos de la clase ilícita de los cuales aprender.

H2 (arquitectura). La estabilidad de las explicaciones depende de la arquitectura, porque cada una combina de forma distinta la información de la transacción con la de sus vecinas.

H3 (compensación). La técnica de compensación del desbalance no cambia la estabilidad de las explicaciones, porque modifica cuánto pesa cada clase en la función de pérdida pero no la información disponible para el modelo.

1.5. Objetivos

1.5.1. Objetivo general

Evaluar la estabilidad de los métodos de explicabilidad aplicados a redes neuronales de grafos entrenadas para detectar lavado de dinero en el conjunto de datos Elliptic, bajo distintos niveles de desbalance de clases, arquitecturas y técnicas de compensación, con el fin de establecer qué factores la determinan y en qué medida sus explicaciones son confiables para una auditoría.

1.5.2. Objetivos específicos

1. Medir el efecto del nivel de desbalance de clases, desde 1:10 hasta 1:200, sobre la estabilidad de las explicaciones de cinco explicadores: GNNExplainer, PGExplainer, valores de Shapley por muestreo, Integrated Gradients y Expected Gradients.

2. Comparar la estabilidad de las explicaciones entre cuatro arquitecturas de GNN: GCN, GraphSAGE, GAT y TAGCN.

3. Analizar el efecto de tres técnicas de compensación del desbalance (sin ajuste, pesos por clase y *focal loss*) y del sobremuestreo con SMOTE sobre el rendimiento predictivo y sobre la estabilidad de las explicaciones.

4. Determinar, mediante un control con pesos aleatorios y medidas de acuerdo que no dependen de las variables nulas, qué parte de la estabilidad medida proviene del modelo entrenado y qué parte de los datos de entrada y del explicador.

Respecto de la propuesta aprobada, dos métodos se sustituyeron por alternativas que se ajustan mejor al problema. GNNShap se reemplazó por valores de Shapley estimados por muestreo sobre las variables de la transacción [18], porque la pregunta de la tesis es qué variables pesan y GNNShap atribuye importancia a las aristas. GraphSMOTE se reemplazó por SMOTE sobre los atributos [12], porque la estructura del grafo de Elliptic no se modifica en ningún escenario. Integrated Gradients [19] y Expected Gradients [20] se añadieron como explicadores de referencia, uno determinista y uno estocástico, que no requieren entrenamiento.

1.6. Contribuciones

1. Un diseño factorial replicado (4 arquitecturas, 6 escenarios de desbalance, 3 técnicas de compensación y 3 semillas de modelo, 216 modelos) que mide la estabilidad de cinco explicadores sobre las mismas 30 transacciones ilícitas.

2. La evidencia de que la partición temporal habitual de Elliptic deja la prueba completa después del cierre del mercado ilegal y vuelve inútil la evaluación en prueba, junto con una partición alternativa que la recupera.

3. La identificación de un artefacto de medida: cerca de la mitad de las variables de una transacción valen cero después del escalado, y los explicadores las dejan siempre al final del orden de importancia, lo que infla la correlación de Spearman entre explicaciones. La tesis cuantifica ese efecto con dos medidas que no dependen de esas variables y con un control de pesos aleatorios [21].

1.7. Alcance y limitaciones

El estudio se limita al conjunto de datos Elliptic y a la tarea de clasificar transacciones como ilícitas o no ilícitas. Las transacciones sin etiqueta se tratan como no ilícitas, que es el supuesto más cercano a la operación real (Capítulo 4). La estabilidad se mide sobre explicaciones de variables, salvo en PGExplainer, que solo explica aristas. No se evalúa la plausibilidad de las explicaciones frente a un patrón real de lavado, porque Elliptic no publica el significado de sus variables ni anotaciones de tipologías; un grafo sintético construido con ese fin se resume en el Anexo B. No se contempla el despliegue del sistema ni su evaluación con analistas.

1.8. Estructura del documento

El Capítulo 2 presenta los fundamentos de las GNN, del desbalance de clases y de los métodos de explicabilidad usados. El Capítulo 3 revisa los trabajos sobre detección de lavado con grafos y sobre la evaluación de explicaciones. El Capítulo 4 describe los datos, la partición temporal, los escenarios, el entrenamiento, los explicadores, las medidas de estabilidad y las pruebas estadísticas. El Capítulo 5 presenta los resultados y contrasta las hipótesis. El Capítulo 6 interpreta los resultados y sus límites, y el Capítulo 7 resume las respuestas a las preguntas de investigación. Los anexos recogen la corrida con la partición original, el eje sintético y los detalles de reproducibilidad.

Capítulo 2

Marco teórico

2.1. Lavado de dinero en criptomonedas

El lavado de dinero es el proceso por el cual se oculta el origen ilícito de unos fondos para integrarlos en la economía formal. La literatura lo describe en tres etapas: colocación, en la que los fondos entran al sistema; estratificación, en la que se mueven a través de muchas operaciones para borrar su rastro; e integración, en la que vuelven a la economía como fondos aparentemente legítimos [1]. Las criptomonedas ofrecen a esas etapas un medio rápido, transfronterizo y seudónimo, y han dado lugar a técnicas propias como la mezcla de fondos, el encadenamiento de transacciones y el uso de plataformas de intercambio con controles débiles [22, 23].

Los sistemas tradicionales de monitoreo se basan en reglas fijas, como umbrales de monto o listas de contrapartes, y producen una proporción muy alta de falsas alarmas [2]. El aprendizaje automático permite aprender patrones a partir de casos etiquetados [1, 24, 25]. En Bitcoin, la cadena de bloques es pública y las transacciones están enlazadas por los fondos que se transfieren, de modo que el problema se puede plantear como la clasificación de los nodos de un grafo [9].

2.2. Redes neuronales de grafos

Sea un grafo con N nodos, matriz de atributos $\mathbf{X} \in \mathbb{R}^{N \times d}$ y conjunto de vecinos $\mathcal{N}(v)$ de cada nodo v. Una red neuronal de grafos calcula en cada capa ℓ una representación $\mathbf{h}_v^{(\ell)}$ de cada nodo combinando su propia representación con las de sus vecinos [3, 4, 26]:

$$\mathbf{h}_v^{(\ell+1)} = \phi\left(\mathbf{h}_v^{(\ell)}, \bigoplus_{u \in \mathcal{N}(v)} \psi\left(\mathbf{h}_v^{(\ell)}, \mathbf{h}_u^{(\ell)}\right)\right), \qquad \mathbf{h}_v^{(0)} = \mathbf{x}_v, \tag{2.1}$$

donde $\bigoplus$ es una agregación invariante al orden (suma, media o máximo) y ϕ y ψ son funciones con parámetros aprendidos. Tras L capas, la representación de un nodo depende solo de los nodos a L saltos o menos, su subgrafo receptivo. La última capa produce la probabilidad de cada clase. Las cuatro arquitecturas del estudio difieren en cómo combinan el nodo con sus vecinos.

GCN [27]. Promedia al nodo con sus vecinos usando la matriz de adyacencia normalizada por el grado, con un único conjunto de pesos:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\mathbf{W}^{(\ell)} \sum_{u \in \mathcal{N}(v)\cup\{v\}} \frac{\mathbf{h}_u^{(\ell)}}{\sqrt{(d_u+1)(d_v+1)}}\right). \tag{2.2}$$

El nodo entra en la suma como un vecino más y no tiene un peso propio.

GraphSAGE [28]. Separa la representación del nodo de la de sus vecinos con dos matrices de pesos:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\mathbf{W}_1^{(\ell)}\mathbf{h}_v^{(\ell)} + \mathbf{W}_2^{(\ell)} \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(\ell)}\right). \tag{2.3}$$

GAT [29]. Pondera a cada vecino, incluido el propio nodo, con un coeficiente de atención aprendido:

$$\mathbf{h}_v^{(\ell+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)\cup\{v\}} \alpha_{vu}\mathbf{W}^{(\ell)}\mathbf{h}_u^{(\ell)}\right), \qquad \alpha_{vu} = \mathrm{softmax}_u\Big(\mathrm{LeakyReLU}\Big(\mathbf{a}^\top[\mathbf{W}\mathbf{h}_v \,\|\, \mathbf{W}\mathbf{h}_u]\Big)\Big), \tag{2.4}$$

y repite el cálculo con varias cabezas de atención independientes cuyos resultados se concatenan.

TAGCN [30]. Aplica un filtro polinomial de orden K sobre la matriz de adyacencia normalizada $\tilde{\mathbf{A}}$, con una matriz de pesos por cada potencia:

$$\mathbf{H}^{(\ell+1)} = \sigma\left(\sum_{k=0}^{K} \tilde{\mathbf{A}}^k \mathbf{H}^{(\ell)} \mathbf{W}_k^{(\ell)}\right). \tag{2.5}$$

El término $k = 0$ da al nodo un peso propio, y cada capa alcanza vecinos a K saltos.

2.3. Desbalance de clases

Cuando una clase es mucho más frecuente que otra, la pérdida promedio está dominada por la clase mayoritaria y el modelo puede minimizarla prediciendo casi siempre esa clase [10, 11]. Las estrategias para compensarlo actúan sobre los datos o sobre la pérdida.

Sobre los datos, se puede submuestrear la clase mayoritaria o sobremuestrear la minoritaria.

SMOTE [12] genera ejemplos sintéticos de la clase minoritaria interpolando entre un ejemplo y uno de sus vecinos más cercanos de la misma clase: $\tilde{\mathbf{x}} = \mathbf{x}_i + \lambda(\mathbf{x}_j - \mathbf{x}_i)$, con $\lambda \sim \mathscr{U}(0,1)$.

Sobre la pérdida, sea p_i la probabilidad que el modelo asigna a la clase verdadera del ejemplo i. La entropía cruzada sin ajuste es $\mathscr{L} = -\frac{1}{n}\sum_i \log p_i$. Con pesos por clase, cada ejemplo se pondera por el inverso de la frecuencia de su clase, $w_c = n/(2n_c)$, de modo que

$$\mathscr{L}_{\mathrm{PC}} = \frac{1}{2}\frac{1}{n_+}\sum_{i:\,y_i=1}(-\log p_i) + \frac{1}{2}\frac{1}{n_-}\sum_{i:\,y_i=0}(-\log p_i), \tag{2.6}$$

y cada clase aporta la mitad de la pérdida sin importar cuántos ejemplos tenga. La *focal loss* [13] reduce el peso de los ejemplos que el modelo ya clasifica bien:

$$\mathscr{L}_{\mathrm{FL}} = -\frac{1}{n}\sum_i \alpha_{y_i}(1-p_i)^{\gamma}\log p_i, \tag{2.7}$$

donde γ controla cuánto se atenúan los ejemplos fáciles y α_{y_i} pondera la clase. Con $\alpha = 0{,}75$ para la clase rara y 0,25 para la frecuente, un ejemplo ilícito pesa tres veces lo que uno negativo, mientras que con pesos por clase y una razón de 41 a 1 pesa 41 veces más.

Métricas bajo desbalance. La exactitud no sirve: con un 2 % de ilícitas, predecir siempre la clase negativa da un 98 % de aciertos. La curva ROC relaciona la tasa de verdaderos positivos con la de falsos positivos, y como esta última se divide por el número enorme de negativos, puede verse alta aunque las falsas alarmas superen con creces a los aciertos. La curva de precisión y exhaustividad sí refleja ese costo, y su área (PR-AUC) tiene como referencia del azar la prevalencia de la clase positiva [31]. El coeficiente de correlación de Matthews resume la matriz de confusión completa y es más informativo que el F1 bajo desbalance [32].

2.4. Explicabilidad de modelos

La explicabilidad busca que las decisiones de un modelo puedan ser comprendidas por una persona [33, 34, 7]. Los métodos post hoc explican un modelo ya entrenado sin modificarlo, y los locales explican una predicción individual asignando una importancia a cada elemento de la entrada. En grafos, esos elementos pueden ser las variables de los nodos, las aristas o subgrafos completos [8, 35]. Esta tesis estudia explicaciones de variables: para una transacción, cuál de sus 165 variables pesó más en la predicción.

GNNExplainer [36]. Busca una máscara $\mathbf{M}$ sobre las variables (y opcionalmente sobre las aristas) que preserve la predicción del modelo y sea a la vez pequeña:

$$\min_{\mathbf{M}} \; -\log P(\hat{y} \mid \mathbf{X} \odot \sigma(\mathbf{M})) + \lambda_1 \|\sigma(\mathbf{M})\|_1 + \lambda_2 H(\sigma(\mathbf{M}))\,, \tag{2.8}$$

donde H es la entropía de la máscara, que la empuja hacia valores cercanos a 0 o 1. La máscara se inicializa al azar y se optimiza por descenso de gradiente, por lo que dos ejecuciones pueden terminar en máscaras distintas.

PGExplainer [37]. Entrena una red que, a partir de las representaciones de los dos extremos de una arista, predice su importancia. Una vez entrenada, explica cualquier nodo sin optimizar de nuevo. Atribuye importancia a aristas, no a variables.

Valores de Shapley [38, 18]. Tratan a las variables como jugadores de un juego cooperativo cuyo pago es la predicción. La importancia de la variable j es su contribución marginal promedio sobre todos los órdenes posibles en que las variables se incorporan:

$$\varphi_j = \frac{1}{|\Pi|} \sum_{\pi \in \Pi} \left[f\left(S_\pi^j \cup \{j\}\right) - f\left(S_\pi^j\right) \right], \tag{2.9}$$

donde S_π^j es el conjunto de variables que preceden a j en el orden π y las variables ausentes toman un valor de referencia. Como hay $d!$ órdenes, se estima con una muestra aleatoria de permutaciones [39], y por ello es estocástico.

Integrated Gradients [19]. Integra el gradiente de la salida a lo largo del camino recto entre una referencia $\mathbf{x}'$ y la entrada $\mathbf{x}$:

$$\mathrm{IG}_j(\mathbf{x}) = (x_j - x'_j) \int_0^1 \frac{\partial f(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}'))}{\partial x_j} \, d\alpha. \tag{2.10}$$

La integral se aproxima con una suma de Riemann. Con una referencia fija, el método es determinista. Si la referencia es el vector cero y una variable vale cero, su atribución es exactamente cero.

Expected Gradients [20]. Sustituye la referencia fija por una distribución de referencias D tomadas de los datos y promedia: $\mathrm{EG}_j(\mathbf{x}) = \mathbb{E}_{\mathbf{x}' \sim D,\, \alpha \sim \mathscr{U}(0,1)} \left[(x_j - x'_j)\, \partial_j f(\mathbf{x}' + \alpha(\mathbf{x} - \mathbf{x}')) \right]$. La esperanza se estima por muestreo, por lo que es estocástico.

2.5. Evaluación de las explicaciones

No existe una única medida de la calidad de una explicación. Las revisiones sistemáticas distinguen varias propiedades [16, 17]: la **fidelidad**, si la explicación refleja lo que de verdad usa el modelo; la **plausibilidad**, si coincide con lo que un experto considera relevante; y la **estabilidad** o consistencia, si explicaciones de la misma predicción en condiciones equivalentes se parecen entre sí. En grafos, las evaluaciones sistemáticas han mostrado que los explicadores difieren mucho entre sí y que su desempeño depende del conjunto de datos y de la arquitectura [14, 15].

La estabilidad es una condición necesaria para usar una explicación como evidencia: si dos ejecuciones del mismo explicador sobre el mismo modelo y la misma transacción dan órdenes de importancia distintos, el analista no sabe a cuál creer. Pero no es suficiente. Un explicador que ignorara el modelo y devolviera siempre el mismo orden sería perfectamente estable e inútil. Adebayo et al. [21] propusieron por eso comparar la explicación de un modelo entrenado con la de la misma arquitectura con pesos aleatorios: si se parecen, la explicación no depende de lo aprendido.

Para medir el acuerdo entre dos explicaciones se usan comparaciones de rangos o de conjuntos. La correlación de Spearman [40] compara los órdenes completos de importancia, y el índice de Jaccard [41] compara los conjuntos de las k variables más importantes. También se han propuesto métricas diseñadas específicamente para la estabilidad de las explicaciones [42].

Capítulo 3

Estado del arte

Este capítulo revisa tres líneas de trabajo que la tesis cruza: la detección de lavado de dinero con grafos, en particular sobre Elliptic (Sección 3.1); la explicabilidad aplicada a ese problema (Sección 3.2); y la evaluación de la estabilidad de las explicaciones, en especial bajo desbalance de clases (Sección 3.3). Al final se precisa el vacío que aborda el trabajo (Sección 3.4).

3.1. Detección de lavado de dinero con grafos

Las revisiones sobre aprendizaje automático para la prevención del lavado de dinero documentan el paso de sistemas basados en reglas a modelos supervisados y no supervisados [1], y, más recientemente, a modelos de aprendizaje profundo [2]. Una revisión sistemática dedicada a la detección de fraude financiero con GNN muestra que esta familia de modelos se ha extendido a tarjetas de crédito, seguros, transferencias bancarias y criptomonedas [5].

Elliptic se publicó junto con los primeros experimentos de GCN para este problema [9]. Sus autores compararon regresión logística, bosques aleatorios y GCN, y observaron que el desempeño en la prueba se desplomaba a partir del paso 43, tras el cierre de un mercado ilegal. Ese hallazgo es la base de la partición que usa esta tesis. Desde entonces el conjunto de datos se ha convertido en la referencia para la detección de transacciones ilícitas en Bitcoin. Sobre él se han propuesto modelos de *gradient boosting* [24]; modelos que combinan GCN con memorias de largo y corto plazo para aprovechar el orden temporal, junto con esquemas de aprendizaje activo [43]; representaciones de nodos aprendidas de forma autosupervisada [44]; y arquitecturas temporales basadas en transformadores sobre grafos [45]. Otras propuestas modelan el grafo de transacciones como una secuencia de instantáneas dinámicas [46, 47]. En el ámbito forense, Pocher et al. analizan cómo los métodos de aprendizaje automático y de análisis del grafo de transacciones se aplican a la prevención del lavado en criptomonedas y qué implicaciones tiene su uso por autoridades y empresas [25].

Dos observaciones de esta literatura son relevantes para la tesis. La primera es que la mayoría de los trabajos reporta el desempeño con la partición temporal habitual o con particiones aleatorias, y pocos discuten el efecto del cierre del mercado sobre la prueba. La segunda es que el desbalance de clases se trata como un obstáculo para el rendimiento. Lawal et al. examinan explícitamente las limitaciones de las GNN sobre Elliptic bajo ese desbalance, y muestran que un modelo puede lograr una exactitud alta en entrenamiento sin identificar de forma significativa las transacciones ilícitas, con y sin estrategias como los pesos por clase [48]. Ninguno de estos trabajos mide qué ocurre con las explicaciones cuando cambia el desbalance.

3.2. Explicabilidad en la detección de lavado de dinero

La necesidad de explicar las alertas de un sistema de detección se justifica tanto por la regulación como por la operación: el analista debe poder entender y documentar por qué revisa una transacción [6, 2]. Una revisión reciente reúne los trabajos sobre detección de lavado en criptomonedas que incorporan explicabilidad [49], y se han propuesto marcos de GNN explicables para detectar transacciones financieras ilícitas [50]. En la mayoría de estos trabajos la explicación se presenta como un resultado cualitativo, por ejemplo las variables o los vecinos más importantes para algunos casos, y no se evalúa si la explicación se mantiene al repetirla.

En el campo general de la explicabilidad de GNN, las taxonomías clasifican los métodos según lo que explican (variables, aristas o subgrafos) y cómo lo obtienen (gradientes, perturbaciones, sustitutos o descomposición) [8, 35]. Las evaluaciones comparativas han mostrado que los explicadores discrepan entre sí. Agarwal et al. construyeron un generador de grafos sintéticos con explicaciones de referencia, capaz de variar el tamaño, la distribución de grados y la homofilia del grafo, y una biblioteca para evaluar explicadores sobre esos grafos y sobre grafos reales [14]. Longa et al. probaron 12 explicadores sobre 8 arquitecturas de paso de mensajes y 6 conjuntos de datos, y analizaron por qué unas arquitecturas resultan más explicables que otras y qué explicador conviene en cada caso [15].

3.3. Estabilidad de las explicaciones y desbalance de clases

Las revisiones sobre la evaluación cuantitativa de la explicabilidad incluyen la estabilidad, o consistencia, entre las propiedades que una explicación debe cumplir, junto a la fidelidad y la plausibilidad [16, 17]. Se han propuesto métricas específicas para medirla [42]. Adebayo et al. mostraron que algunas explicaciones visualmente convincentes apenas cambian cuando se aleatorizan los pesos del modelo, y propusieron esa aleatorización como prueba de que una explicación depende de lo que el modelo aprendió [21].

La relación entre desbalance de clases y explicaciones se ha estudiado en datos tabulares. Rai et al. diseñaron un marco comparativo para evaluar la confiabilidad y la consistencia de LIME, SHAP y gráficos de dependencia parcial en la predicción de riesgo de enfermedades con historias clínicas electrónicas, bajo distintos grados de desbalance [51]. Fujiwara et al. proponen combinar destilación de conocimiento y remuestreo para mejorar tanto el rendimiento como la estabilidad de las explicaciones en clasificación desbalanceada [52]. Estos trabajos indican que el desbalance y las técnicas que lo compensan pueden afectar las explicaciones, pero no tratan datos en grafo ni explicadores de GNN.

3.4. Vacío de investigación

De la revisión se desprende que (1) los trabajos sobre Elliptic se concentran en el rendimiento y tratan la explicación de forma cualitativa; (2) las evaluaciones sistemáticas de explicadores de GNN usan sobre todo grafos sintéticos o de otros dominios y no varían el desbalance; y (3) los estudios sobre desbalance y explicaciones usan datos tabulares. No se encontró un estudio que mida la estabilidad de varios explicadores de GNN sobre un problema real de lavado de dinero mientras se controlan, en un diseño factorial con réplicas independientes, el nivel de desbalance, la arquitectura y la técnica de compensación. Tampoco se encontró uno que verifique, con un control de pesos aleatorios, si la estabilidad medida en ese problema refleja lo que el modelo aprendió. Esta tesis aborda ambas carencias.

Capítulo 4

Metodología

Este capítulo describe el diseño experimental completo: los datos y sus etiquetas (Sección 4.1), la partición temporal y la razón para excluir el periodo del cierre del mercado (Sección 4.2), los escenarios de desbalance (Sección 4.3), el entrenamiento de los modelos (Sección 4.4), los explicadores (Sección 4.5), las medidas de estabilidad (Sección 4.6), el control de pesos aleatorios (Sección 4.7) y las pruebas estadísticas (Sección 4.8). La Tabla 4.1 resume los factores.

Tabla 4.1: Factores del diseño experimental. Cada combinación se entrena con tres semillas de modelo, lo que da $4 \times 6 \times 3 \times 3 = 216$ modelos.

Factor	Niveles
Arquitectura	GCN, GraphSAGE, GAT, TAGCN
Escenario de desbalance	Nativo (1:41,1), 1:10, 1:20, 1:10 con SMOTE, 1:100, 1:200
Compensación	Sin ajuste, pesos por clase, *focal loss*
Semilla del modelo	42, 43, 44
Explicador	GNNExplainer, PGExplainer, Shapley por muestreo, Integrated Gradients, Expected Gradients
Medida de estabilidad	Spearman sobre las 165 variables (principal), Spearman sobre las variables distintas de cero, Jaccard de las 10 variables principales

4.1. Datos

Se usa el conjunto de datos Elliptic [9], un grafo de 203.769 transacciones de Bitcoin unidas por 234.355 aristas dirigidas, que representan flujos de fondos entre transacciones. Cada transacción tiene 165 variables numéricas anonimizadas: 93 describen la propia transacción (montos, número de entradas y salidas, comisiones y otras) y 72 agregan información de sus vecinas a uno y dos saltos. Además, cada transacción pertenece a uno de 49 pasos de tiempo, separados aproximadamente dos semanas entre sí, y no hay aristas entre pasos distintos. Los autores no publican el significado de las variables.

Las etiquetas son tres: 4.545 transacciones ilícitas (2,2 %), 42.019 lícitas (20,6 %) y 157.205 sin etiqueta (77,1 %). Como clase positiva se toma la ilícita. Como clase negativa se toman las lícitas y las sin etiqueta juntas. Esta elección reproduce la situación de un sistema real, en el que la inmensa mayoría de las transacciones no ha sido revisada por nadie y no se puede descartar que haya ilícitas entre ellas. Con ella, la razón entre negativas e ilícitas en el entrenamiento es de 41 a 1, que es el desbalance real del problema. La alternativa de usar solo las lícitas como negativas da una razón de 7,6 a 1 y produce un problema más fácil y menos realista.

Las variables se escalan con la mediana y el rango intercuartílico, ajustados solo con las transacciones del periodo de entrenamiento, para que los valores extremos de los montos no dominen. Una consecuencia de este escalado, que resulta importante para la medición de la estabilidad, es que toda variable igual a su mediana queda exactamente en cero. En las 30 transacciones que se explican (Sección 4.5), en promedio el 44 % de las variables vale cero, y la mediana de variables distintas de cero por transacción es 89 de 165.

4.2. Partición temporal y exclusión del cierre del mercado

La partición respeta el orden del tiempo: el modelo se entrena con los pasos más antiguos y se evalúa con los más recientes, de modo que nunca ve el futuro. La partición habitual en la literatura usa los pasos 1 a 34 para entrenar y los pasos 35 a 49 para evaluar, y en este trabajo se dividió esa evaluación en validación (35 a 42) y prueba (43 a 49).

En el paso 43 se cerró un mercado ilegal importante, y los autores de Elliptic reportaron que desde ese momento ningún modelo detectaba las nuevas transacciones ilícitas [9]. Es un cambio en la distribución de los datos a lo largo del tiempo [53]: lo que el modelo aprendió de los mercados que existían deja de describir la actividad ilícita posterior. Con la partición habitual, la prueba completa cae después del cierre. En la corrida de esta tesis con esa partición, el mejor modelo de cada arquitectura obtuvo en prueba un PR-AUC entre 0,005 y 0,012, frente a 0,006 que obtendría un clasificador al azar (Anexo A).

Por ello el estudio principal usa solo los pasos 1 a 42 y los reparte en tres bloques contiguos: entrenamiento con los pasos 1 a 29, validación con los pasos 30 a 35 y prueba con los pasos 36 a 42 (Figura 4.1 y Tabla 4.2). Los pasos 43 a 49 no se usan.

La partición tiene tres costos que conviene declarar. Primero, el entrenamiento dispone de 2.871 transacciones ilícitas en lugar de 3.462, es decir, 591 menos que con la partición habitual. Segundo, los cortes en los pasos 29 y 35 los escogieron los autores para que validación y prueba tuvieran cada una cerca de una sexta parte de las ilícitas anteriores al cierre; no responden a un criterio externo. Tercero, la prueba ya no mide la capacidad del modelo para adaptarse a un cambio drástico del mercado, sino su desempeño en un periodo posterior al entrenamiento pero

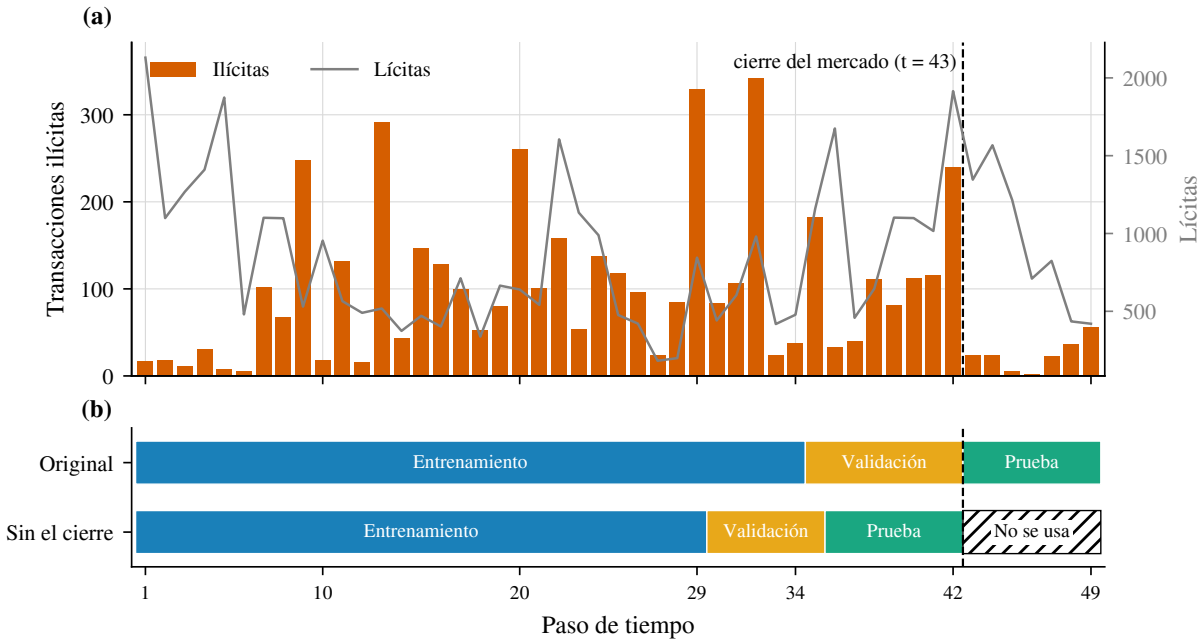


Figura 4.1: (a) Transacciones ilícitas (barras) y lícitas (línea) por paso de tiempo en Elliptic. La línea discontinua marca el cierre del mercado ilegal en el paso 43. (b) Partición original y partición sin el cierre usada en el estudio principal; en esta última los pasos 43 a 49 no se usan.

Tabla 4.2: Transacciones por parte de cada partición. Negativas son lícitas más sin etiqueta; la razón es negativas por cada ilícita.

Partición	Parte	Pasos	Ilícitas	Negativas	Razón
Original	Entrenamiento	1 a 34	3.462	132.803	1:38,4
	Validación	35 a 42	914	36.906	1:40,4
	Prueba	43 a 49	169	29.515	1:174,6
Sin el cierre	Entrenamiento	1 a 29	2.871	117.933	1:41,1
	Validación	30 a 35	773	20.195	1:26,1
	Prueba	36 a 42	732	31.581	1:43,1

en condiciones parecidas. Esta tesis no estudia la adaptación a ese cambio, sino la estabilidad de las explicaciones, y para ello necesita modelos que detecten de verdad.

4.3. Escenarios de desbalance

El nivel de desbalance se modifica solo en el entrenamiento; validación y prueba conservan siempre su proporción real. Los escenarios principales protegen la señal del fraude: no quitan transacciones ilícitas, sino que submuestrean las negativas o sobremuestrean las ilícitas. Las negativas se submuestrean de forma estratificada, conservando la misma mezcla de lícitas y sin etiqueta que en el escenario nativo, con una semilla de datos fija (2026) que es distinta de la semilla del modelo. Submuestrear significa sacar transacciones de la función de pérdida, no del grafo: las transacciones descartadas siguen transmitiendo información a sus vecinas.

- **Nativo (1:41,1).** Las 2.871 ilícitas y las 117.933 negativas del entrenamiento.
- **1:10 y 1:20.** Las 2.871 ilícitas y 28.710 o 57.420 negativas.
- **1:10 con SMOTE.** Las 2.871 ilícitas, 2.871 ilícitas sintéticas y 57.420 negativas. Cada sintética interpola los atributos de una ilícita y de una de sus vecinas más cercanas en el espacio de atributos [12], se añade al grafo en el mismo paso de tiempo y hereda las aristas de la ilícita de la que parte.
- **1:100 y 1:200.** Escenarios de estrés solicitados por el director para forzar el desbalance más allá del real. Como el nativo ya usa todas las negativas, la única forma de llegar a esas razones es quitar ilícitas: se conservan 1.179 y 589 ilícitas, con las 117.933 negativas.

Los escenarios de estrés cambian dos cosas a la vez, la proporción y la cantidad absoluta de ejemplos ilícitos, por lo que sus resultados se analizan junto a los principales pero se interpretan con esa salvedad.

4.4. Modelos y entrenamiento

Se entrenan cuatro arquitecturas de GNN para clasificar nodos: GCN [27], GraphSAGE [28], GAT [29] y TAGCN [30], descritas en la Sección 2.2. Todas reciben el grafo completo y producen una probabilidad de que cada transacción sea ilícita.

Compensación del desbalance. Se comparan tres funciones de pérdida (Sección 2.3): entropía cruzada sin ajuste; entropía cruzada con pesos por clase inversamente proporcionales a su frecuencia en el entrenamiento, de modo que cada clase aporta la mitad de la pérdida total; y *focal loss* [13] con $\gamma = 2$ y $\alpha = 0{,}75$ como peso de la clase ilícita.

Búsqueda de hiperparámetros. Para cada combinación de arquitectura, escenario y pérdida se hace una búsqueda bayesiana de 8 ensayos que maximiza el PR-AUC de validación. El primer ensayo usa valores tomados de la literatura y los siguientes se proponen con un estimador de Parzen. Cada ensayo entrena hasta 50 épocas, con parada temprana a las 10 épocas sin mejora. El espacio es el mismo para las cuatro arquitecturas: dimensión oculta en $\{64, 128, 148\}$, 2 o 3 capas, *dropout* entre 0,1 y 0,5, tasa de aprendizaje entre 10^{-4} y 10^{-2} y decaimiento de pesos entre 10^{-5} y 10^{-3}, las dos últimas en escala logarítmica. GAT busca además 2 o 4 cabezas de atención y TAGCN un orden de filtro K entre 2 y 4. Por memoria, GAT se limita a 2 capas: con 148 dimensiones y 8 cabezas el grafo completo no cabe en una GPU de 8 GB. Todas las arquitecturas reciben el mismo número de ensayos y de épocas, para que la comparación no confunda arquitectura con presupuesto de cómputo.

Entrenamiento final y semillas. Con los mejores hiperparámetros se entrena el modelo final hasta 150 épocas con Adam, con parada temprana a las 20 épocas sin mejora del F1 de validación. Esto se repite con tres semillas de modelo (42, 43 y 44); las semillas 43 y 44 reutilizan los hiperparámetros encontrados con la 42, de modo que la diferencia entre semillas refleja solo la aleatoriedad del entrenamiento. El cálculo en GPU no es determinista a nivel de pesos, porque las sumas de mensajes usan operaciones atómicas, lo que se suma a esa aleatoriedad.

Evaluación del rendimiento. La métrica principal es el PR-AUC, el área bajo la curva de precisión y exhaustividad, porque bajo un desbalance fuerte el ROC-AUC puede ser alto aunque el modelo produzca muchas más falsas alarmas que aciertos [31]. Se reporta también el ROC-AUC. El umbral de decisión se fija en la validación, como el que maximiza el F1, sin usar información de la prueba.

Filtro de calidad. Explicar un modelo que no aprendió no informa sobre el problema. Por eso cada modelo se marca como aprobado si en validación, con umbral 0,5, alcanza un F1 de al menos 0,30 y un coeficiente de Matthews [32] de al menos 0,15. El filtro se usa para interpretar los resultados, no para excluir modelos de la medición: todos los modelos se explican, y los análisis principales se restringen a las arquitecturas que aprenden.

4.5. Explicadores

Transacciones explicadas. Se explican las mismas 30 transacciones para todos los modelos: 30 ilícitas de validación sorteadas con una semilla fija (1234) entre las 773 ilícitas de los pasos 30 a 35. Usar el mismo conjunto permite comparar explicaciones entre modelos. Se explica la validación y no la prueba porque es el periodo con el que se seleccionó el modelo y donde su comportamiento está mejor caracterizado.

Subgrafo receptivo. Una GNN de L capas solo usa, para predecir una transacción, la información de sus vecinas a L saltos o menos (a $L \cdot K$ saltos en TAGCN). Cada explicación se calcula sobre ese subgrafo, y se verifica que la predicción del modelo en el subgrafo coincida con la del grafo completo. Esto reduce la memoria de varios gigabytes a unos pocos megabytes sin cambiar lo que se explica.

Se usan cinco explicadores, descritos en la Sección 2.4:

- **GNNExplainer** [36], que optimiza una máscara sobre las variables de los nodos del subgrafo durante 100 iteraciones con tasa de aprendizaje 0,01. La importancia de cada variable es la media de su máscara sobre los nodos del subgrafo.
- **PGExplainer** [37], que entrena una red que asigna importancia a las aristas. Se entrena durante 100 épocas con 50 nodos, de los cuales al menos 25 son ilícitos. Como explica aristas y no variables, su estabilidad se reporta aparte.

- **Shapley por muestreo** [18, 39], que estima la contribución de cada variable promediando 50 permutaciones aleatorias; una variable ausente toma el valor medio de esa variable en el subgrafo.
- **Integrated Gradients** [19], que integra el gradiente a lo largo del camino recto entre una referencia y la transacción, con 100 pasos. La referencia es la mediana del entrenamiento, que tras el escalado es el vector cero. Es determinista.
- **Expected Gradients** [20], que promedia Integrated Gradients sobre 200 referencias tomadas al azar de 5.000 transacciones negativas del entrenamiento.

Ninguno de los explicadores de variables se entrena previamente: los dos de gradiente y el de Shapley se aplican directamente al modelo, y GNNExplainer optimiza una máscara nueva para cada transacción. Solo PGExplainer tiene una fase de entrenamiento propia.

Réplicas. Cada explicador se ejecuta cinco veces sobre cada transacción y cada modelo, con semillas distintas. La estabilidad mide cuánto se parecen esas cinco explicaciones.

4.6. Medidas de estabilidad

Sea $\mathbf{a}^{(r)} \in \mathbb{R}^{165}$ el vector de importancias que la réplica r asigna a las variables de una transacción. La estabilidad de esa transacción es el promedio, sobre los $\binom{5}{2} = 10$ pares de réplicas, de una medida de acuerdo s:

$$E = \frac{1}{10} \sum_{r<r'} s\left(\mathbf{a}^{(r)}, \mathbf{a}^{(r')}\right), \tag{4.1}$$

y la estabilidad de un modelo es el promedio de E sobre las 30 transacciones. Se usan tres medidas de acuerdo:

Spearman global (principal). La correlación de rangos de Spearman [40] entre los dos vectores completos de 165 importancias. Es la medida más usada en la literatura y permite comparar con otros trabajos.

Spearman sobre las variables distintas de cero. La misma correlación, calculada solo sobre las variables cuyo valor en la transacción es distinto de cero.

Jaccard top-10. El índice de Jaccard [41] entre los conjuntos de las 10 variables más importantes de cada réplica, $|A \cap B|/|A \cup B|$. Si las dos réplicas comparten k de sus 10 variables, el índice vale $k/(20-k)$: compartir 3, 5 o 7 variables da 0,18, 0,33 y 0,54.

Las dos medidas adicionales responden a un artefacto que se detectó durante el estudio. Una variable que vale cero recibe una importancia nula o casi nula de los cuatro explicadores de

variables, por razones distintas en cada uno: Integrated Gradients multiplica el gradiente por la diferencia entre la variable y la referencia, que es cero; la máscara de GNNExplainer anula las entradas sin gradiente; y en Shapley y Expected Gradients la referencia suele coincidir con cero. Esas variables, cerca de la mitad, quedan siempre al final del orden en cualquier réplica y en cualquier modelo, y el Spearman global cuenta su acuerdo como si fuera información. Con la partición original se estimó que ese bloque fija por sí solo un piso de Spearman entre 0,74 y 0,82. El Spearman sobre las variables distintas de cero elimina el bloque, y el Jaccard top-10 solo mira las variables que el analista vería primero.

Además de la estabilidad entre réplicas de un mismo modelo, se mide el acuerdo entre las explicaciones medias de dos modelos distintos sobre las mismas transacciones: entre semillas, entre escenarios, entre arquitecturas y entre explicadores.

4.7. Control de pesos aleatorios

Una explicación estable no es necesariamente una explicación del modelo. Si el explicador dependiera solo de los datos de la transacción, daría el mismo resultado con cualquier modelo. Para comprobarlo se aplica la prueba de aleatorización de parámetros de Adebayo et al. [21]: para cada arquitectura y semilla, en el escenario nativo con pesos por clase (12 modelos), se explican las mismas 30 transacciones con el modelo entrenado y con dos redes de la misma arquitectura e hiperparámetros pero con pesos aleatorios, sin entrenar. Se reporta el acuerdo entre la explicación del modelo entrenado y la de la red aleatoria. Si ese acuerdo es tan alto como la estabilidad entre réplicas, el orden de las variables no depende de lo que el modelo aprendió.

4.8. Análisis estadístico

La unidad independiente es el modelo entrenado, es decir, una arquitectura con una semilla. Los modelos de una misma semilla en distintos escenarios parten de los mismos pesos iniciales y de los mismos hiperparámetros, y sus explicaciones resultan casi idénticas (acuerdo de 0,96 entre escenarios con la misma semilla frente a 0,92 entre semillas). Tratarlos como observaciones independientes inflaría la significancia, por lo que las pruebas se construyen así:

H1 (escenario). Prueba de Friedman [54] con 9 bloques (3 arquitecturas que aprenden por 3 semillas) y 6 tratamientos (los escenarios), con pesos por clase. Se complementa con una prueba de equivalencia de dos pruebas unilaterales (TOST) [55] entre el nativo y 1:200 con margen de 0,05: rechazar ambas hipótesis nulas indica que la diferencia, si existe, es menor que el margen.

H2 (arquitectura). Prueba de Friedman con 3 bloques (las semillas) y 4 tratamientos (las arquitecturas), sobre la media de los cuatro escenarios principales con pesos por clase. Con tres bloques la prueba tiene poca potencia, por lo que se reporta además en cuántas semillas queda primera cada arquitectura.

H3 (compensación). Prueba de signos pareada sobre 9 pares (arquitectura por semilla, media de los cuatro escenarios principales), de pesos por clase frente a *focal loss* y frente a sin ajuste.

Los intervalos de confianza de las figuras se obtienen por *bootstrap* sobre los modelos [56]. Todas las pruebas se repiten con las tres medidas de estabilidad, y una conclusión se considera sólida solo si las tres coinciden.

Capítulo 5

Resultados

Este capítulo presenta los resultados de la corrida principal, con la partición sin el cierre del mercado. Primero se muestra que esa partición recupera una prueba informativa (Sección 5.1) y cómo rinden los modelos según el escenario y la pérdida (Sección 5.2). Después se contrastan las tres hipótesis (Secciones 5.3 a 5.5) y se examina qué parte de la estabilidad medida depende del modelo (Sección 5.6). Salvo que se indique otra cosa, cada cifra es la media de tres semillas de modelo.

5.1. La partición sin el cierre recupera la prueba

La Figura 5.1 compara el rendimiento en prueba de las dos particiones, en el escenario nativo y con la pérdida que obtuvo el mayor PR-AUC de validación para cada arquitectura. Con la partición original, cuya prueba cae después del cierre, ningún modelo supera al azar. Sin el cierre, tres de las cuatro arquitecturas detectan: GraphSAGE alcanza un PR-AUC de 0,273, TAGCN de 0,244 y GAT de 0,172, entre 7 y 12 veces el valor del azar (0,023). GCN queda en 0,085. El ROC-AUC sube de valores entre 0,48 y 0,71 a valores entre 0,73 y 0,88. El mejor modelo de toda la matriz, GraphSAGE con pesos por clase en el escenario 1:20, obtiene en prueba un PR-AUC de 0,306 y un ROC-AUC de 0,901.

La Figura 5.2 muestra las curvas completas. La curva de precisión y exhaustividad deja ver lo que el ROC-AUC oculta: con la mitad de las ilícitas recuperadas, la precisión de GraphSAGE y de TAGCN ronda 0,3, es decir, unas dos falsas alarmas por cada acierto. La Tabla 5.1 resume las cifras.

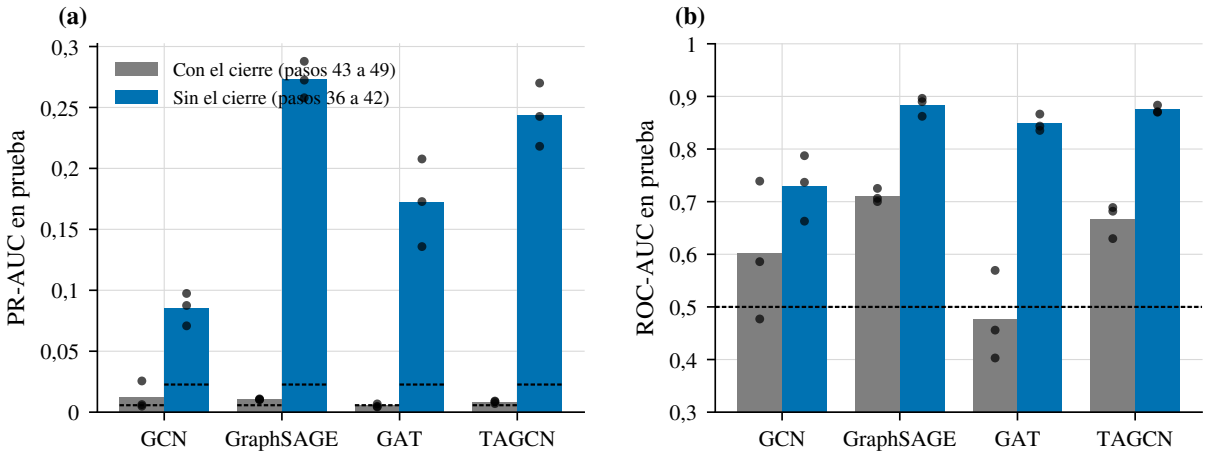


Figura 5.1: Rendimiento en prueba con la partición original (prueba en los pasos 43 a 49, después del cierre) y con la partición sin el cierre (prueba en los pasos 36 a 42). Escenario nativo, pérdida elegida por validación. Las barras son la media de tres semillas, los puntos cada semilla y las líneas discontinuas el valor del azar: la prevalencia en (a) y 0,5 en (b).

Tabla 5.1: Rendimiento en el escenario nativo con la pérdida elegida por validación (media ± desviación estándar de tres semillas). Filtro: modelos que superan el filtro de calidad en validación.

Arquitectura	Pérdida elegida	PR-AUC val.	PR-AUC prueba	ROC-AUC prueba	Filtro
GCN	Focal loss	0,214 ± 0,054	0,085 ± 0,013	0,729 ± 0,063	0/3
GraphSAGE	Focal loss	0,478 ± 0,038	0,273 ± 0,015	0,883 ± 0,018	3/3
GAT	Sin ajuste	0,282 ± 0,027	0,172 ± 0,036	0,848 ± 0,016	2/3
TAGCN	Focal loss	0,529 ± 0,074	0,244 ± 0,026	0,875 ± 0,007	3/3
Azar		0,037	0,023	0,5	

5.2. Rendimiento según el escenario y la pérdida

La Figura 5.3 y la Tabla 5.2 muestran el PR-AUC de validación de cada combinación. Tres regularidades se observan.

La mejor pérdida depende del escenario. En el escenario nativo, *focal loss* iguala o supera a los pesos por clase: en GraphSAGE, 0,478 frente a 0,326; en TAGCN, 0,529 frente a 0,212. En 1:10, la entropía sin ajuste es la mejor en ambas (0,551 y 0,594). No hay una pérdida que sea la mejor en todos los escenarios.

Con poco fraude solo resisten los pesos por clase. En 1:100 y 1:200, la *focal loss* de GraphSAGE cae a 0,189 y 0,054, y la de TAGCN a 0,282 y 0,054. Los pesos por clase mantienen a GraphSAGE en 0,447 y 0,420, y a TAGCN en 0,357 y 0,307. Es el mismo comportamiento que en la partición original.

La entropía sin ajuste es frágil. En GraphSAGE pasa de 0,551 en 1:10 a 0,035 en 1:20 y se queda en el nivel del azar en todos los escenarios más desbalanceados; en TAGCN baja de 0,594 a 0,046 entre 1:10 y 1:100. Cuando el desbalance supera cierto punto, el modelo minimiza la

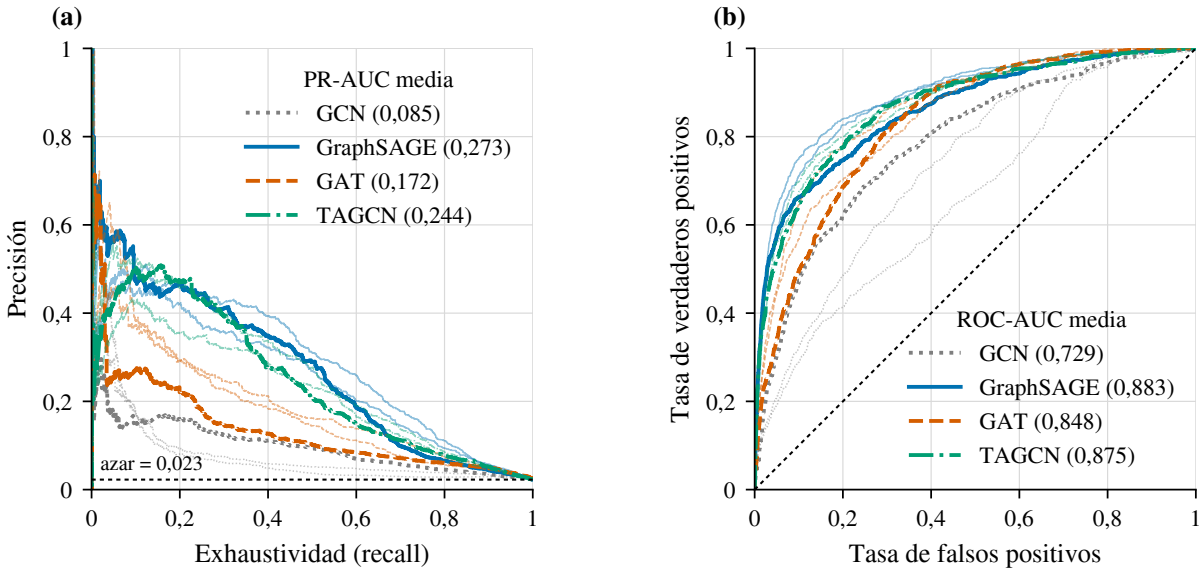


Figura 5.2: Curvas de precisión y exhaustividad (a) y ROC (b) en la prueba de la partición sin el cierre, escenario nativo, con la pérdida elegida por validación para cada arquitectura. Trazo grueso: semilla 42; trazos finos: semillas 43 y 44. Entre paréntesis, el área media de las tres semillas.

pérdida prediciendo casi siempre la clase negativa. GAT es la excepción: es la arquitectura en la que la entropía sin ajuste supera más veces el filtro de calidad (Tabla 5.3).

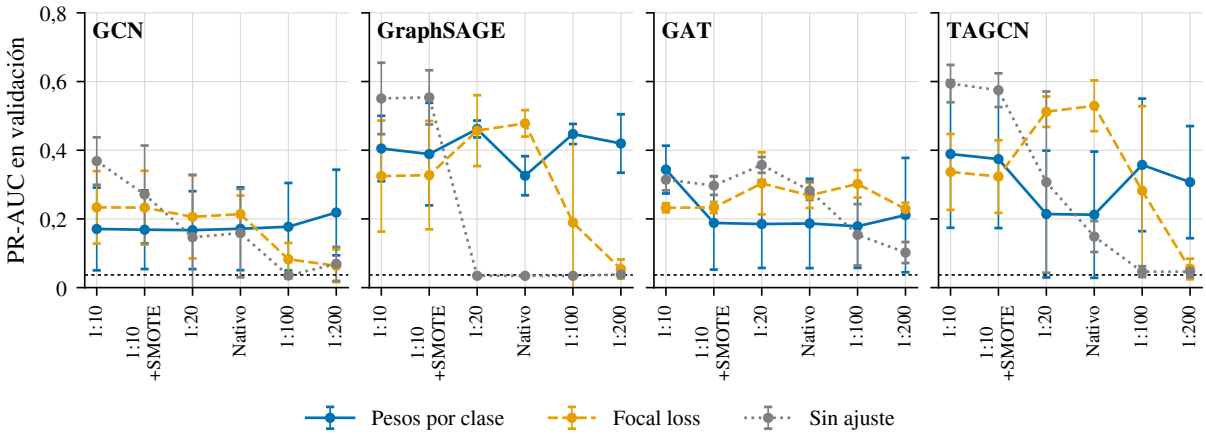


Figura 5.3: PR-AUC de validación por escenario y pérdida (media y desviación estándar de tres semillas). La línea discontinua es el valor del azar (0,037).

GCN no aprende en este problema. Ningún modelo GCN con pesos por clase supera el filtro de calidad (Tabla 5.3). GCN promedia a cada transacción con sus vecinas sin darle un peso propio (Sección 2.2), y en Elliptic la señal está sobre todo en la propia transacción. Por eso los análisis de estabilidad por escenario y por pérdida se restringen a GraphSAGE, GAT y TAGCN, y GCN se incluye solo en la comparación entre arquitecturas, con esa salvedad. Hay que añadir que GAT con pesos por clase tampoco supera el filtro en ningún modelo, aunque su PR-AUC de validación esté entre 0,18 y 0,34, muy por encima del azar: el filtro usa el umbral fijo de 0,5, y con ese umbral su F1 queda entre 0,13 y 0,17. Se mantiene en el análisis porque su PR-AUC

Tabla 5.2: PR-AUC de validación por escenario, arquitectura y pérdida (media de tres semillas).

Arquitectura	Pérdida	1:10	1:10 + SMOTE	1:20	Nativo	1:100	1:200
GCN	Pesos por clase	0,171	0,169	0,167	0,171	0,177	0,219
	Focal loss	0,234	0,233	0,206	0,214	0,083	0,064
	Sin ajuste	0,369	0,271	0,147	0,159	0,035	0,069
GraphSAGE	Pesos por clase	0,405	0,389	0,462	0,326	0,447	0,420
	Focal loss	0,325	0,327	0,457	0,478	0,189	0,054
	Sin ajuste	0,551	0,554	0,035	0,034	0,034	0,038
GAT	Pesos por clase	0,344	0,188	0,185	0,187	0,179	0,211
	Focal loss	0,233	0,233	0,304	0,268	0,302	0,229
	Sin ajuste	0,314	0,297	0,357	0,282	0,154	0,102
TAGCN	Pesos por clase	0,389	0,374	0,214	0,212	0,357	0,307
	Focal loss	0,337	0,323	0,512	0,529	0,282	0,054
	Sin ajuste	0,594	0,575	0,307	0,149	0,046	0,046

muestra que sí ordena las transacciones mejor que el azar.

Tabla 5.3: Modelos que superan el filtro de calidad en validación (F1 $\geq$ 0,30 y MCC $\geq$ 0,15 con umbral 0,5), de 18 por celda (6 escenarios por 3 semillas).

Arquitectura	Pesos por clase	Focal loss	Sin ajuste
GCN	0/18	3/18	6/18
GraphSAGE	10/18	11/18	6/18
GAT	0/18	8/18	11/18
TAGCN	8/18	13/18	7/18

5.3. H1: el nivel de desbalance no cambia la estabilidad

La Figura 5.4 y la Tabla 5.4 muestran la estabilidad entre réplicas de cada explicador en los seis escenarios, con pesos por clase y las tres arquitecturas que aprenden. Con la medida principal, la estabilidad de GNNExplainer se mueve entre 0,917 (nativo) y 0,939 (1:100), la de Shapley entre 0,991 y 0,994 y la de Expected Gradients entre 0,918 y 0,934. Integrated Gradients es determinista y vale 1 en todos los casos. No hay una tendencia a bajar a medida que el desbalance se hace más extremo: 1:100 y 1:200 no son los escenarios menos estables. Las dos medidas robustas confirman la ausencia de tendencia, aunque con valores más bajos (Sección 5.6).

Las pruebas confirman la lectura visual. Con la medida principal, la prueba de Friedman no rechaza la igualdad de los seis escenarios para ningún explicador ($p = 0{,}27$ en GNNExplainer, 0,53 en Shapley y 0,25 en Expected Gradients), y la prueba de equivalencia entre el nativo y 1:200 la acepta en los tres ($p \leq 0{,}0002$): la diferencia entre el escenario real y el más extremo es menor que 0,05. Con las medidas robustas hay dos excepciones que conviene examinar. Con el Jaccard top-10, Friedman rechaza la igualdad en Expected Gradients ($p = 0{,}018$), pero la diferencia entre el mejor y el peor escenario es de 0,03 y el escenario más estable es precisamente 1:200; no hay degradación. Con el Spearman sobre las variables distintas de cero,

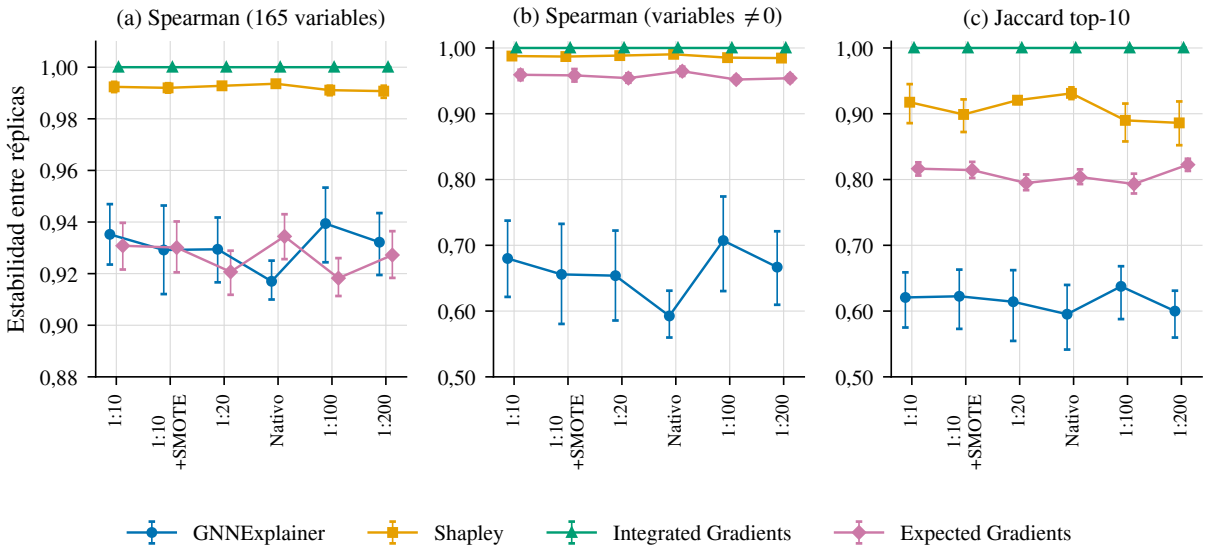


Figura 5.4: Estabilidad entre réplicas por escenario y explicador, con las tres medidas. Pesos por clase; media de 9 modelos (3 arquitecturas que aprenden por 3 semillas) e intervalo *bootstrap* del 95 %.

Tabla 5.4: Estabilidad entre réplicas por escenario (pesos por clase, 9 modelos). p Friedman: igualdad de los seis escenarios. p TOST: equivalencia entre nativo y 1:200 con margen 0,05 (un valor bajo indica equivalencia).

Medida	Explicador	1:10	1:10 + SMOTE	1:20	Nativo	1:100	1:200	p Friedman	p TOST
Spearman global	GNNExplainer	0,935	0,929	0,929	0,917	0,939	0,932	0,269	$< 0{,}001$
	Shapley	0,992	0,992	0,993	0,994	0,991	0,991	0,534	$< 0{,}001$
	Integrated Gradients	1,000	1,000	1,000	1,000	1,000	1,000	–	$< 0{,}001$
	Expected Gradients	0,931	0,930	0,921	0,934	0,918	0,927	0,248	$< 0{,}001$
Spearman $\neq 0$	GNNExplainer	0,680	0,656	0,654	0,593	0,707	0,667	0,117	0,791
	Shapley	0,988	0,987	0,988	0,990	0,985	0,985	0,448	$< 0{,}001$
	Integrated Gradients	1,000	1,000	1,000	1,000	1,000	1,000	–	$< 0{,}001$
	Expected Gradients	0,959	0,958	0,954	0,965	0,952	0,954	0,552	$< 0{,}001$
Jaccard top-10	GNNExplainer	0,621	0,623	0,614	0,595	0,638	0,600	0,233	0,012
	Shapley	0,918	0,899	0,921	0,931	0,890	0,886	0,117	0,394
	Integrated Gradients	1,000	1,000	1,000	1,000	1,000	1,000	–	$< 0{,}001$
	Expected Gradients	0,816	0,814	0,795	0,804	0,793	0,823	0,018	$< 0{,}001$

GNNExplainer no alcanza la equivalencia ($p = 0{,}79$): sus valores van de 0,593 (nativo) a 0,707 (1:100), con un intervalo amplio, pero de nuevo el escenario menos estable es el nativo y no los más desbalanceados.

Veredicto sobre H1. La hipótesis no se sostiene. En ninguno de los cuatro explicadores ni con ninguna de las tres medidas la estabilidad disminuye a medida que el desbalance se hace más extremo, ni siquiera en 1:100 y 1:200, donde además se quitaron transacciones ilícitas del entrenamiento. El mismo resultado se obtuvo con la partición original (Anexo A).

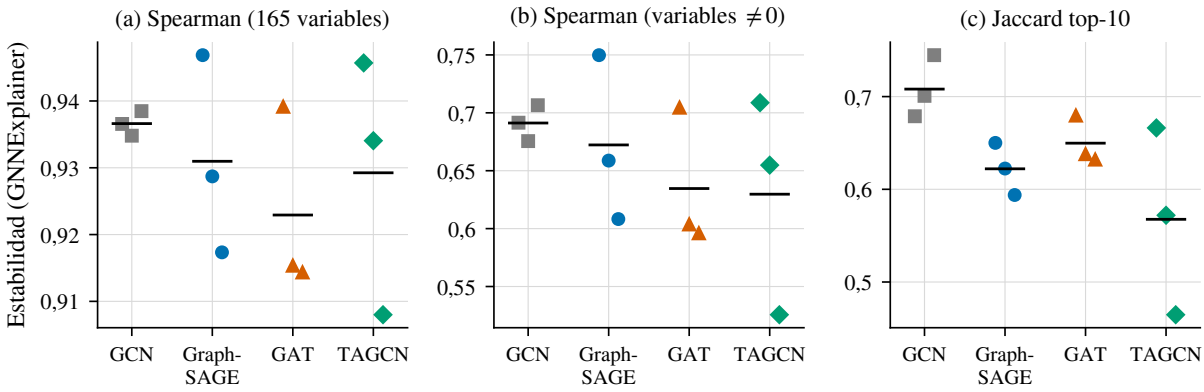


Figura 5.5: Estabilidad de GNNExplainer por arquitectura con las tres medidas (pesos por clase, media de los cuatro escenarios principales). Cada punto es una semilla; la línea horizontal es la media.

Tabla 5.5: Estabilidad por arquitectura (pesos por clase, media de los escenarios principales, 3 semillas). p (4): Friedman con las cuatro arquitecturas; p (3): sin GCN. Primera: número de semillas en que cada arquitectura obtiene el mayor valor.

Medida	Explicador	GCN	GraphSAGE	GAT	TAGCN	p (4)	p (3)	Primera
Spearman global	GNNExplainer	0,937	0,931	0,923	0,929	0,532	0,264	GCN 2, GraphSAGE 1
	Shapley	0,996	0,994	0,994	0,991	0,042	0,097	GCN 3
	Expected Gradients	0,923	0,935	0,932	0,920	0,172	0,097	GraphSAGE 2, GAT 1
Spearman $\neq 0$	GNNExplainer	0,691	0,672	0,635	0,630	0,334	0,097	GCN 2, GraphSAGE 1
	Shapley	0,993	0,991	0,990	0,985	0,042	0,097	GCN 3
	Expected Gradients	0,958	0,965	0,960	0,952	0,457	0,264	GAT 1, GraphSAGE 1, GCN 1
Jaccard top-10	GNNExplainer	0,708	0,622	0,650	0,568	0,060	0,097	GCN 2, GAT 1
	Shapley	0,948	0,929	0,927	0,896	0,122	0,264	GCN 2, GraphSAGE 1
	Expected Gradients	0,798	0,800	0,816	0,806	0,532	0,264	GAT 2, GCN 1

5.4. H2: no hay un orden estable entre arquitecturas

Con GNNExplainer, las cuatro arquitecturas quedan dentro de dos centésimas con la medida principal (de 0,923 en GAT a 0,937 en GCN), y las semillas de una misma arquitectura se dispersan tanto como las arquitecturas entre sí (Figura 5.5). Ninguna prueba de Friedman rechaza la igualdad con GNNExplainer (p entre 0,06 y 0,53) ni con Expected Gradients, y sin GCN ninguna lo hace con ningún explicador ni medida ($p \geq 0{,}097$). La única diferencia significativa aparece con Shapley y las cuatro arquitecturas ($p = 0{,}042$): GCN es la más estable en las tres semillas, pero la diferencia con la última es de cinco milésimas (0,996 frente a 0,991). Con tres semillas como bloques la prueba tiene poca potencia, por lo que la ausencia de diferencias no prueba la igualdad; lo que sí muestra es que no hay un orden que se repita.

Un resultado llama la atención: GCN, la arquitectura que no aprende, obtiene las explicaciones más estables con GNNExplainer en dos de las tres semillas y en las tres medidas. Esto anticipa lo que muestra la Sección 5.6: una explicación estable no es necesariamente una explicación de algo aprendido.

Veredicto sobre H2. La hipótesis no se sostiene entre las arquitecturas que aprenden: Graph-

SAGE, GAT y TAGCN producen explicaciones con estabilidad indistinguible. En la partición original GraphSAGE había resultado la más estable en las tres semillas; ese orden no se repite sin el cierre, y por tanto no se puede afirmar.

5.5. H3: la técnica de compensación no cambia la estabilidad

Tabla 5.6: Estabilidad por técnica de compensación (3 arquitecturas que aprenden, media de los escenarios principales). PC > FL y PC > SA: en cuántos de los 9 pares de arquitectura y semilla pesos por clase supera a *focal loss* o a sin ajuste, con el p de la prueba de signos.

Medida	Explicador	Pesos por clase	Focal loss	Sin ajuste	PC > FL	PC > SA
Spearman global	GNNExplainer	0,928	0,922	0,930	6.0/9 (0,508)	4.0/9 (1,000)
	Shapley	0,993	0,995	0,994	0.0/9 (0,004)	5.0/9 (1,000)
	Expected Gradients	0,929	0,933	0,938	2.0/9 (0,180)	1.0/9 (0,039)
Spearman $\neq 0$	GNNExplainer	0,646	0,626	0,668	5.0/9 (1,000)	2.0/9 (0,180)
	Shapley	0,988	0,992	0,990	1.0/9 (0,039)	4.0/9 (1,000)
	Expected Gradients	0,959	0,960	0,962	4.0/9 (1,000)	4.0/9 (1,000)
Jaccard top-10	GNNExplainer	0,613	0,598	0,625	5.0/9 (1,000)	5.0/9 (1,000)
	Shapley	0,917	0,941	0,922	2.0/9 (0,180)	5.0/9 (1,000)
	Expected Gradients	0,807	0,799	0,805	7.0/9 (0,180)	4.0/9 (1,000)

Con GNNExplainer, ninguna comparación es significativa con ninguna medida: pesos por clase supera a *focal loss* en 6 de los 9 pares con la medida principal ($p = 0{,}51$) y a la entropía sin ajuste en 4 ($p = 1$), y las diferencias medias son menores que 0,025 con las tres medidas. Con Shapley y Expected Gradients aparecen diferencias sistemáticas pero minúsculas: con Shapley, *focal loss* es más estable que pesos por clase en los 9 pares ($p = 0{,}004$), por dos milésimas; con Expected Gradients, la entropía sin ajuste supera a pesos por clase en 8 de 9 ($p = 0{,}039$), por nueve milésimas. El sentido de la diferencia cambia según el explicador, y en la corrida original con GNNExplainer era el contrario (Anexo A).

Veredicto sobre H3. La hipótesis se sostiene en términos prácticos: la técnica de compensación cambia mucho el rendimiento (Sección 5.2) pero mueve la estabilidad de los explicadores de variables en milésimas o pocas centésimas, sin un sentido constante.

Tabla 5.7: Estabilidad de PGExplainer sobre las aristas del subgrafo: Spearman entre réplicas y Jaccard de las 20 aristas principales (escenarios principales, media de 3 semillas).

	Pesos por clase		Focal loss		Sin ajuste	
Arquitectura	Spearman	Jaccard	Spearman	Jaccard	Spearman	Jaccard
GCN	0,517	0,630	0,325	0,645	0,322	0,608
GraphSAGE	0,251	0,560	0,064	0,628	0,381	0,658
GAT	0,305	0,621	0,514	0,625	0,458	0,675
TAGCN	0,504	0,769	0,121	0,628	0,288	0,650

PGExplainer. PGExplainer explica aristas y no variables, por lo que se analiza aparte (Tabla 5.7). El grafo de Elliptic es muy disperso: la mediana de aristas en el subgrafo receptivo de

las transacciones explicadas es de 2 o 3, y la estabilidad solo se pudo medir, en promedio, en 13 de las 30 transacciones (las demás tienen menos aristas de las necesarias para comparar órdenes). Con tan pocas aristas, el Spearman entre réplicas es una medida muy gruesa y el Jaccard de las 20 principales compara casi siempre conjuntos que incluyen todas las aristas disponibles. Aun así, el Spearman va de 0,06 a 0,52 según la combinación, sin patrón por arquitectura ni por pérdida. La conclusión es que, en este problema, PGExplainer no produce una explicación de aristas evaluable, y que la variación entre pérdidas no puede atribuirse a la técnica de compensación.

5.6. Qué parte de la estabilidad depende del modelo

Las secciones anteriores muestran que la estabilidad apenas cambia con el escenario, la arquitectura o la pérdida. Hay dos explicaciones posibles: que los modelos aprendan casi lo mismo en todas las condiciones, o que la estabilidad medida no dependa de lo aprendido. El control de pesos aleatorios (Sección 4.7) permite distinguirlas (Figura 5.6 y Tabla 5.8).

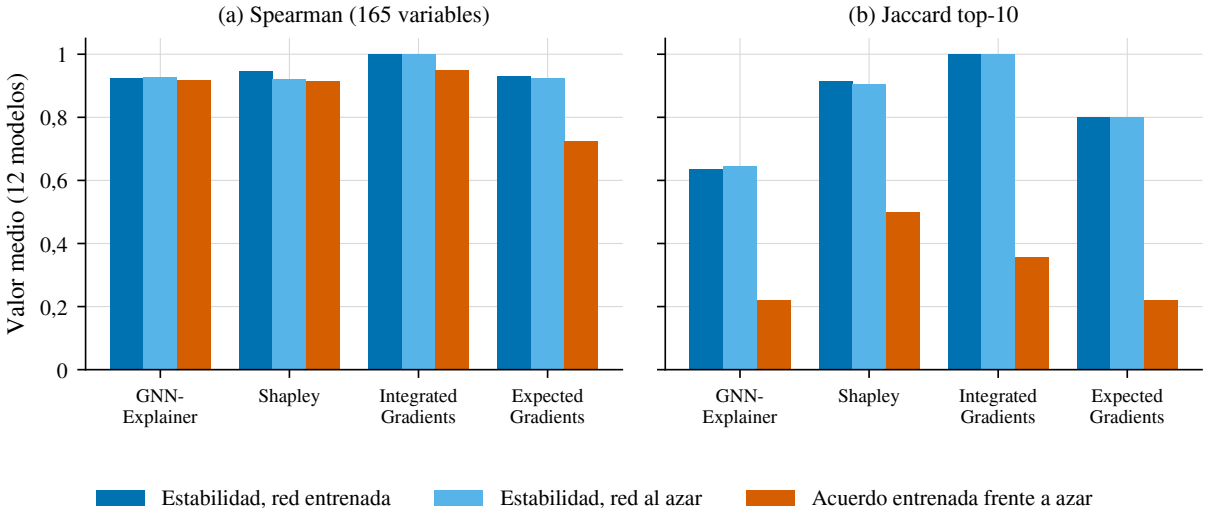


Figura 5.6: Control de pesos aleatorios en el escenario nativo con pesos por clase (12 modelos: 4 arquitecturas por 3 semillas). Para cada explicador, la estabilidad entre réplicas con la red entrenada y con una red de pesos aleatorios, y el acuerdo entre la explicación de la red entrenada y la de la red aleatoria.

Tabla 5.8: Control de pesos aleatorios (media de 12 modelos).

	Spearman global			Jaccard top-10		
Explicador	Estab. entrenada	Estab. azar	Entrenada vs azar	Estab. entrenada	Estab. azar	Entrenada vs azar
GNNExplainer	0,924	0,927	0,917	0,636	0,646	0,220
Shapley	0,948	0,920	0,914	0,915	0,904	0,498
Integrated Gradients	1,000	1,000	0,950	1,000	1,000	0,357
Expected Gradients	0,932	0,924	0,724	0,801	0,799	0,220

Con la medida principal, la explicación no depende del modelo. La explicación de GNNExplainer sobre el modelo entrenado coincide con la de una red sin entrenar con un Spearman de 0,917, prácticamente lo mismo que coinciden dos réplicas sobre el mismo modelo (0,924). Lo mismo ocurre con Shapley (0,914 frente a 0,948) e Integrated Gradients (0,950). Una red con pesos aleatorios, que no aprendió nada, recibe además explicaciones igual de estables que la entrenada (0,927 en GNNExplainer). El orden de Integrated Gradients coincide con el orden de las variables por su valor absoluto con un Spearman de 0,97. La medida principal está dominada por el bloque de variables en cero (Sección 4.6), que ocupa las mismas posiciones del orden con cualquier red.

Con el Jaccard top-10, sí depende. Las 10 variables principales de la red entrenada y de la aleatoria comparten en promedio un índice de 0,22 con GNNExplainer y Expected Gradients, 0,36 con Integrated Gradients y 0,50 con Shapley, muy por debajo de la estabilidad entre réplicas (0,64, 0,80, 1 y 0,92). Las variables que el analista vería primero sí cambian con los pesos del modelo. Expected Gradients es el único explicador cuyo acuerdo con la red aleatoria también es claramente menor que su estabilidad con la medida principal (0,724 frente a 0,932), porque sus referencias no son el vector cero y su atribución no se anula en las variables nulas.

Modelos igual de buenos dan explicaciones distintas. El acuerdo entre explicaciones de modelos distintos refuerza esta lectura. Con GNNExplainer y pesos por clase, dos semillas de la misma arquitectura y el mismo escenario coinciden con un Spearman global de 0,929, pero con un Jaccard top-10 de solo 0,30: comparten unas 5 de sus 10 variables principales. Cambiar de escenario manteniendo la semilla mueve menos la explicación (Jaccard 0,60) que cambiar de semilla manteniendo el escenario. La principal fuente de variación de la explicación no es el desbalance, sino la aleatoriedad del entrenamiento.

Los explicadores no coinciden entre sí. Sobre el mismo modelo, los cuatro explicadores de variables señalan conjuntos de variables principales muy distintos (Figura 5.7): con el Jaccard top-10 el acuerdo va de 0,10 (GNNExplainer frente a Shapley) a 0,49 (Integrated frente a Expected Gradients). El alto Spearman global entre GNNExplainer e Integrated Gradients (0,84) desaparece con el Jaccard top-10 (0,29), lo que indica que su parecido se debe sobre todo al bloque de variables nulas.

5.7. Síntesis

La Tabla 5.9 resume el contraste de las hipótesis.

Los tres veredictos tienen una lectura común que el control de pesos aleatorios aclara: con la medida principal, la estabilidad refleja sobre todo la estructura de los datos de entrada y el explicador, no lo que el modelo aprendió, y por eso apenas responde a los factores del diseño.

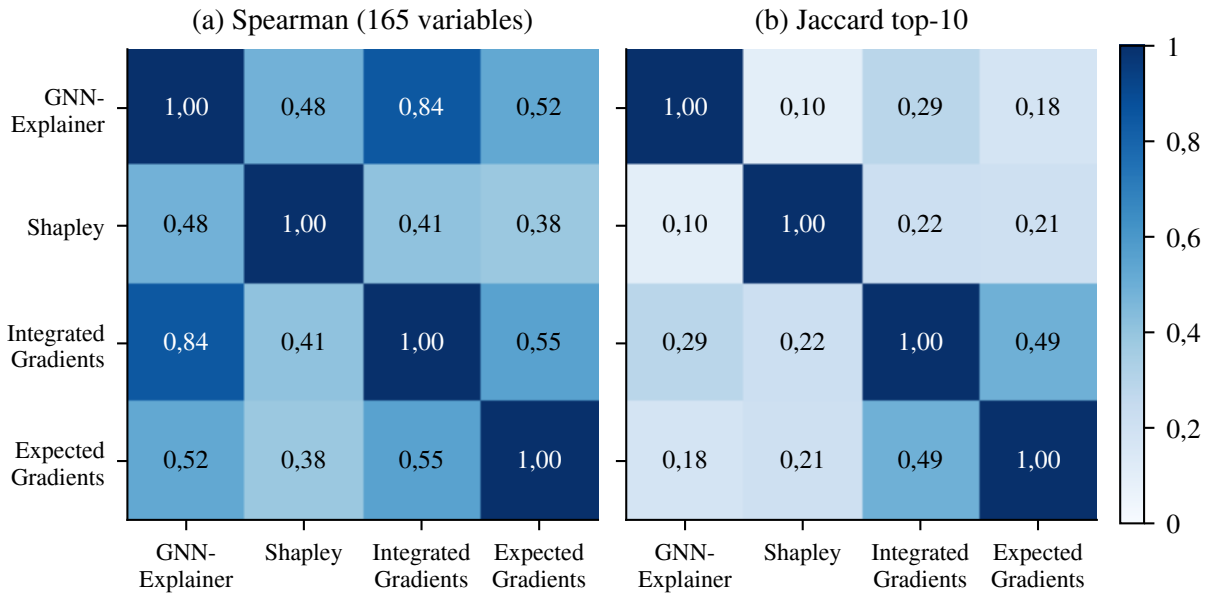


Figura 5.7: Acuerdo entre explicadores sobre el mismo modelo (GraphSAGE, GAT y TAGCN; escenarios principales; todas las pérdidas y semillas), con el Spearman global (a) y el Jaccard top-10 (b).

Tabla 5.9: Veredicto de las hipótesis con la partición sin el cierre.

Hipótesis	Resultado	Veredicto
H1: la estabilidad baja con el desbalance	Sin tendencia en 4 explicadores y 3 medidas; equivalencia entre nativo y 1:200 con la medida principal	No se sostiene
H2: la estabilidad depende de la arquitectura	Sin orden repetido entre GraphSAGE, GAT y TAGCN; GCN, que no aprende, es la más estable	No se sostiene
H3: la compensación no cambia la estabilidad	Diferencias de milésimas o pocas centésimas, con sentido que cambia según el explicador	Se sostiene en términos prácticos

Las 10 variables principales, en cambio, sí dependen del modelo, y varían más entre semillas que entre escenarios.

Capítulo 6

Discusión

[Por redactar después del Capítulo 5.]

Capítulo 7

Conclusiones

[Por redactar después del Capítulo 5.]

Referencias

[1] Zhiyuan Chen, Le Dinh Van Khoa, Ee Na Teoh, Amril Nazir, Ettikan Kandasamy Karuppiah y Kim Sim Lam. "Machine learning techniques for anti-money laundering (AML) solutions in suspicious transaction detection: a review". En: *Knowledge and Information Systems* 57.2 (2018), págs. 245-285. DOI: 10.1007/s10115-017-1144-z.

[2] Dattatray Vishnu Kute, Biswajeet Pradhan, Nagesh Shukla y Abdullah Alamri. "Deep Learning and Explainable Artificial Intelligence Techniques Applied for Detecting Money Laundering–A Critical Review". En: *IEEE Access* 9 (2021), págs. 82300-82317. DOI: 10.1109/access.2021.3086230.

[3] F. Scarselli, M. Gori, Ah Chung Tsoi, M. Hagenbuchner y G. Monfardini. "The Graph Neural Network Model". En: *IEEE Transactions on Neural Networks* 20.1 (2009), págs. 61-80. DOI: 10.1109/tnn.2008.2005605.

[4] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang y Philip S. Yu. "A Comprehensive Survey on Graph Neural Networks". En: *IEEE Transactions on Neural Networks and Learning Systems* 32.1 (2021), págs. 4-24. DOI: 10.1109/tnnls.2020.2978386.

[5] Soroor Motie y Bijan Raahemi. "Financial fraud detection using graph neural networks: A systematic review". En: *Expert Systems with Applications* 240 (2024), pág. 122156. DOI: 10.1016/j.eswa.2023.122156.

[6] Cynthia Rudin. "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead". En: *Nature Machine Intelligence* 1.5 (2019), págs. 206-215. DOI: 10.1038/s42256-019-0048-x.

[7] Alejandro Barredo Arrieta, Natalia Díaz-Rodríguez, Javier Del Ser et al. "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI". En: *Information Fusion* 58 (2020), págs. 82-115. DOI: 10.1016/j.inffus.2019.12.012.

[8] Hao Yuan, Haiyang Yu, Shurui Gui y Shuiwang Ji. "Explainability in Graph Neural Networks: A Taxonomic Survey". En: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2022), págs. 1-19. DOI: 10.1109/tpami.2022.3204236.

[9] Mark Weber, Giacomo Domeniconi, Jie Chen et al. *Anti-Money Laundering in Bitcoin: Experimenting with Graph Convolutional Networks for Financial Forensics*. KDD Workshop on Anomaly Detection in Finance. 2019. arXiv: 1908.02591. URL: https://arxiv.org/abs/1908.02591.

[10] Haibo He y E.A. Garcia. “Learning from Imbalanced Data”. En: *IEEE Transactions on Knowledge and Data Engineering* 21.9 (2009), págs. 1263-1284. DOI: 10.1109/tkde.2008.239.

[11] Justin M. Johnson y Taghi M. Khoshgoftaar. “Survey on deep learning with class imbalance”. En: *Journal of Big Data* 6.1 (2019), pág. 27. DOI: 10.1186/s40537-019-0192-5.

[12] N. V. Chawla, K. W. Bowyer, L. O. Hall y W. P. Kegelmeyer. “SMOTE: Synthetic Minority Oversampling Technique”. En: *Journal of Artificial Intelligence Research* 16 (2002), págs. 321-357. DOI: 10.1613/jair.953.

[13] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He y Piotr Dollar. “Focal Loss for Dense Object Detection”. En: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.2 (2020), págs. 318-327. DOI: 10.1109/tpami.2018.2858826.

[14] Chirag Agarwal, Owen Queen, Himabindu Lakkaraju y Marinka Zitnik. “Evaluating explainability for graph neural networks”. En: *Scientific Data* 10.1 (2023), pág. 144. DOI: 10.1038/s41597-023-01974-x.

[15] Antonio Longa, Steve Azzolin, Gabriele Santin et al. “Explaining the Explainers in Graph Neural Networks: a Comparative Study”. En: *ACM Computing Surveys* 57.5 (2025), págs. 1-37. DOI: 10.1145/3696444.

[16] Meike Nauta, Jan Trienes, Shreyasi Pathak et al. “From Anecdotal Evidence to Quantitative Evaluation Methods: A Systematic Review on Evaluating Explainable AI”. En: *ACM Computing Surveys* 55.13s (2023), págs. 1-42. DOI: 10.1145/3583558.

[17] Francesco Bodria, Fosca Giannotti, Riccardo Guidotti, Francesca Naretto, Dino Pedreschi y Salvatore Rinzivillo. “Benchmarking and survey of explanation methods for black box models”. En: *Data Mining and Knowledge Discovery* 37.5 (2023), págs. 1719-1778. DOI: 10.1007/s10618-023-00933-9.

[18] Erik Štrumbelj e Igor Kononenko. “Explaining prediction models and individual predictions with feature contributions”. En: *Knowledge and Information Systems* 41.3 (2014), págs. 647-665. DOI: 10.1007/s10115-013-0679-x.

[19] Mukund Sundararajan, Ankur Taly y Qiqi Yan. “Axiomatic Attribution for Deep Networks”. En: *Proceedings of the 34th International Conference on Machine Learning (ICML)*. Vol. 70. Proceedings of Machine Learning Research. 2017, págs. 3319-3328. URL: https://proceedings.mlr.press/v70/sundararajan17a.html.

[20] Gabriel Erion, Joseph D. Janizek, Pascal Sturmfels, Scott M. Lundberg y Su-In Lee. “Improving performance of deep learning models with axiomatic attribution priors and expected gradients”. En: *Nature Machine Intelligence* 3.7 (2021), págs. 620-631. DOI: 10.1038/s42256-021-00343-w.

[21] Julius Adebayo, Justin Gilmer, Michael Muelly, Ian Goodfellow, Moritz Hardt y Been Kim. “Sanity Checks for Saliency Maps”. En: *Advances in Neural Information Processing Systems 31 (NeurIPS)*. 2018, págs. 9505-9515. URL: https://papers.nips.cc/paper/8160-sanity-checks-for-saliency-maps.

[22] Hugo Almeida, Pedro Pinto y Ana Fernández Vilas. “From Placement to Integration: A Parametric Study of Cryptocurrency-Based Money Laundering Techniques”. En: *Risks* 13.12 (2025), pág. 249. DOI: 10.3390/risks13120249.

[23] Roland Subashi. “Cryptocurrencies and Money Laundering”. En: *Balkan Journal of Interdisciplinary Research* 10.1 (2024), págs. 55-62. DOI: 10.2478/bjir-2024-0005.

[24] Dylan Vassallo, Vincent Vella y Joshua Ellul. “Application of Gradient Boosting Algorithms for Anti-money Laundering in Cryptocurrencies”. En: *SN Computer Science* 2.3 (2021), pág. 143. DOI: 10.1007/s42979-021-00558-z.

[25] Nadia Pocher, Mirko Zichichi, Fabio Merizzi, Muhammad Zohaib Shafiq y Stefano Ferretti. “Detecting anomalous cryptocurrency transactions: An AML/CFT application of machine learning-based forensics”. En: *Electronic Markets* 33.1 (2023), pág. 37. DOI: 10.1007/s12525-023-00654-3.

[26] Jie Zhou, Ganqu Cui, Shengding Hu et al. “Graph neural networks: A review of methods and applications”. En: *AI Open* 1 (2020), págs. 57-81. DOI: 10.1016/j.aiopen.2021.01.001.

[27] Thomas N. Kipf y Max Welling. “Semi-Supervised Classification with Graph Convolutional Networks”. En: *International Conference on Learning Representations (ICLR)*. 2017. arXiv: 1609.02907. URL: https://arxiv.org/abs/1609.02907.

[28] William L. Hamilton, Rex Ying y Jure Leskovec. “Inductive Representation Learning on Large Graphs”. En: *Advances in Neural Information Processing Systems 30 (NeurIPS)*. 2017, págs. 1024-1034. URL: https://papers.nips.cc/paper/6703-inductive-representation-learning-on-large-graphs.

[29] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò y Yoshua Bengio. “Graph Attention Networks”. En: *International Conference on Learning Representations (ICLR)*. 2018. arXiv: 1710.10903. URL: https://arxiv.org/abs/1710.10903.

[30] Jian Du, Shanghang Zhang, Guanhang Wu, José M. F. Moura y Soummya Kar. *Topology Adaptive Graph Convolutional Networks*. 2017. arXiv: 1710.10370. URL: https://arxiv.org/abs/1710.10370.

[31] Takaya Saito y Marc Rehmsmeier. “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets”. En: *PLOS ONE* 10.3 (2015), e0118432. DOI: 10.1371/journal.pone.0118432.

[32] Davide Chicco y Giuseppe Jurman. “The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation”. En: *BMC Genomics* 21.1 (2020), pág. 6. DOI: 10.1186/s12864-019-6413-7.

[33] Amina Adadi y Mohammed Berrada. “Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI)”. En: *IEEE Access* 6 (2018), págs. 52138-52160. DOI: 10.1109/access.2018.2870052.

[34] Riccardo Guidotti, Anna Monreale, Salvatore Ruggieri, Franco Turini, Fosca Giannotti y Dino Pedreschi. “A Survey of Methods for Explaining Black Box Models”. En: *ACM Computing Surveys* 51.5 (2019), págs. 1-42. DOI: 10.1145/3236009.

[35] Enyan Dai, Tianxiang Zhao, Huaisheng Zhu et al. “A Comprehensive Survey on Trustworthy Graph Neural Networks: Privacy, Robustness, Fairness, and Explainability”. En: *Machine Intelligence Research* 21.6 (2024), págs. 1011-1061. DOI: 10.1007/s11633-024-1510-8.

[36] Rex Ying, Dylan Bourgeois, Jiaxuan You, Marinka Zitnik y Jure Leskovec. “GNNExplainer: Generating Explanations for Graph Neural Networks”. En: *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 2019, págs. 9240-9251. URL: https://papers.nips.cc/paper/9123-gnnexplainer-generating-explanations-for-graph-neural-networks.

[37] Dongsheng Luo, Wei Cheng, Dongkuan Xu et al. “Parameterized Explainer for Graph Neural Network”. En: *Advances in Neural Information Processing Systems 33 (NeurIPS)*. 2020, págs. 19620-19631. URL: https : / / proceedings . neurips . cc / paper / 2020 / hash / e37b08dd3015330dcbb5d6663667b8b8-Abstract.html.

[38] Scott M. Lundberg y Su-In Lee. “A Unified Approach to Interpreting Model Predictions”. En: *Advances in Neural Information Processing Systems 30 (NeurIPS)*. 2017, págs. 4765-4774. URL: https://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions.

[39] Javier Castro, Daniel Gómez y Juan Tejada. “Polynomial calculation of the Shapley value based on sampling”. En: *Computers & Operations Research* 36.5 (2009), págs. 1726-1730. DOI: 10.1016/j.cor.2008.04.004.

[40] C. Spearman. “The Proof and Measurement of Association between Two Things”. En: *The American Journal of Psychology* 15.1 (1904), pág. 72. DOI: 10.2307/1412159.

[41] Paul Jaccard. “THE DISTRIBUTION OF THE FLORA IN THE ALPINE ZONE. 1”. En: *New Phytologist* 11.2 (1912), págs. 37-50. DOI: 10.1111/j.1469-8137.1912.tb05611.x.

[42] Falko Gawantka, Franz Just, Marina Savelyeva, Markus Wappler y Jörg Lässig. “A Novel Metric for Evaluating the Stability of XAI Explanations”. En: *Advances in Science, Technology and Engineering Systems Journal* 9 (2024), págs. 133-142. DOI: 10.25046/aj090113.

[43] Ismail Alarab y Simant Prakoonwit. “Graph-Based LSTM for Anti-money Laundering: Experimenting Temporal Graph Convolutional Network with Bitcoin Data”. En: *Neural Processing Letters* 55.1 (2023), págs. 689-707. DOI: 10.1007/s11063-022-10904-8.

[44] Wai Weng Lo, Gayan K. Kulatilleke, Mohanad Sarhan, Siamak Layeghy y Marius Portmann. “Inspection-L: self-supervised GNN node embeddings for money laundering detection in bitcoin”. En: *Applied Intelligence* 53.16 (2023), págs. 19406-19417. DOI: 10.1007/s10489-023-04504-9.

[45] Ziqian Lin, Qining Luo, Dongze Wu et al. “Detecting illicit transactions in bitcoin: a wavelet-temporal graph transformer approach for anti-money laundering”. En: *Scientific Reports* 16.1 (2026), pág. 1548. DOI: 10.1038/s41598-025-23901-3.

[46] Fei Wan y Ping Li. “A Novel Money Laundering Prediction Model Based on a Dynamic Graph Convolutional Neural Network and Long Short-Term Memory”. En: *Symmetry* 16.3 (2024), pág. 378. DOI: 10.3390/sym16030378.

[47] Yulong Wang, Qingxiao Zheng, Xuedong Li, Lingfeng Wang y Ling Lin. “CoSemiGNN: Blockchain fraud detection with dynamic graph neural networks based on co-association of semi-supervised”. En: *Expert Systems with Applications* 298 (2026), pág. 129853. DOI: 10.1016/j.eswa.2025.129853.

[48] Oluwatosin Lawal, Awele Okolie y Callistus Obunadike. “AN EXPLAINABLE GRAPH NEURAL NETWORK FRAMEWORK FOR ANTI–MONEY LAUNDERING IN CRYPTOCURRENCY TRANSACTIONS USING THE ELLIPTIC DATASET”. En: *International Journal of Network Security & Its Applications* 17.6 (2025), págs. 27-39. DOI: 10.5121/ijnsa.2025.17602.

[49] Ekta Unagar y Bhavesh Borisaniya. “Survey on Detection of Cryptocurrency Money Laundering and Its Explanation Using XAI”. En: *SN Computer Science* 6.3 (2025), pág. 291. DOI: 10.1007/s42979-025-03828-2.

[50] Xiaosong He, Jie Huang, Kai Ma, Hongjing He y Min Li. “An explainable graph neural network framework for illicit financial transaction detection”. En: *Applied Intelligence* 56.4 (2026), pág. 102. DOI: 10.1007/s10489-026-07138-9.

[51] Teena Rai, Jun He, Jaspreet Kaur et al. “Evaluating XAI techniques under class imbalance using CPRD data”. En: *Frontiers in Artificial Intelligence* 8 (2025), pág. 1682919. DOI: 10.3389/frai.2025.1682919.

[52] Kazuki Fujiwara. “Knowledge distillation with resampling for imbalanced data classification: Enhancing predictive performance and explainability stability”. En: *Results in Engineering* 24 (2024), pág. 103406. DOI: 10.1016/j.rineng.2024.103406.

[53] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy y Abdelhamid Bouchachia. “A survey on concept drift adaptation”. En: *ACM Computing Surveys* 46.4 (2014), págs. 1-37. DOI: 10.1145/2523813.

[54] Milton Friedman. “The Use of Ranks to Avoid the Assumption of Normality Implicit in the Analysis of Variance”. En: *Journal of the American Statistical Association* 32.200 (1937), págs. 675-701. DOI: 10.1080/01621459.1937.10503522.

[55] Daniël Lakens. “Equivalence Tests”. En: *Social Psychological and Personality Science* 8.4 (2017), págs. 355-362. DOI: 10.1177/1948550617697177.

[56] B. Efron. “Bootstrap Methods: Another Look at the Jackknife”. En: *The Annals of Statistics* 7.1 (1979). DOI: 10.1214/aos/1176344552.

Apéndice A

Corrida con la partición original

Antes de excluir el periodo del cierre del mercado se ejecutó el estudio completo con la partición habitual de Elliptic: entrenamiento con los pasos 1 a 34 (3.462 ilícitas y 132.803 negativas, razón 1:38,4), validación con los pasos 35 a 42 y prueba con los pasos 43 a 49. Se entrenaron 252 modelos: las mismas 4 arquitecturas, 3 pérdidas y 3 semillas, con los escenarios nativo, 1:10, 1:20, 1:10 con SMOTE, 1:1 (que se apartó del análisis principal), 1:100 y 1:200. En esta partición los escenarios de estrés conservan 1.328 y 664 ilícitas. Este anexo resume lo que esa corrida muestra y por qué motivó el cambio de partición. Los resultados completos están en `results_v4/`.

A.1. La prueba cae al nivel del azar

En la prueba, después del cierre, la prevalencia de ilícitas es de 0,6 %, que es el PR-AUC de un clasificador al azar. Con la pérdida elegida en validación, en el escenario nativo, el PR-AUC de prueba fue de 0,010 para GraphSAGE, 0,008 para TAGCN, 0,005 para GAT y 0,012 para GCN. El ROC-AUC fue 0,711, 0,667, 0,476 y 0,601. Ningún modelo detecta las transacciones ilícitas posteriores al cierre, lo que reproduce lo reportado por los autores del conjunto de datos [9]. En validación, en cambio, los modelos sí aprendían: GraphSAGE con pesos por clase alcanzaba un PR-AUC de 0,462 frente a 0,037 del azar.

A.2. Lo que se mantiene en las dos particiones

La estabilidad de las explicaciones se mide en validación, por lo que no depende de la prueba, y sus conclusiones generales se repiten en las dos corridas:

- El nivel de desbalance no cambia la estabilidad. Con GNNExplainer, pesos por clase y las tres arquitecturas que aprenden, el Spearman global fue 0,946, 0,947, 0,946 y 0,947

en 1:10, 1:10 con SMOTE, 1:20 y nativo, y 0,940 y 0,938 en 1:100 y 1:200. La prueba de Friedman sobre los seis escenarios no rechazó la igualdad ($p = 0{,}84$).

- GCN es la arquitectura más débil en las dos particiones.
- Con poco fraude (1:100 y 1:200) solo los pesos por clase mantienen la detección.
- El control de pesos aleatorios dio, sobre 12 modelos, un acuerdo entre la explicación del modelo entrenado y la de la red sin entrenar de 0,896 con GNNExplainer, 0,911 con Shapley, 0,956 con Integrated Gradients y 0,688 con Expected Gradients, frente a una estabilidad entre réplicas de 0,949, 0,970, 1 y 0,929.

A.3. Lo que cambia

Tres resultados de la corrida original no se repiten sin el cierre, y por eso no se presentan como conclusiones de la tesis:

- Con la partición original, GraphSAGE fue la arquitectura más estable con GNNExplainer en las tres semillas (Spearman global 0,964 frente a 0,939 de GAT y 0,937 de TAGCN). Sin el cierre ese orden desaparece (Sección 5.4).
- Con la partición original, pesos por clase dio explicaciones de GNNExplainer más estables que *focal loss* en los 9 pares de arquitectura y semilla (prueba de signos, $p = 0{,}004$), pero con Shapley el sentido se invertía. Sin el cierre la diferencia con GNNExplainer tampoco se sostiene (Sección 5.5).
- En validación, pesos por clase era la mejor pérdida en el escenario nativo. Sin el cierre, *focal loss* la iguala o la supera en ese escenario.

Una advertencia metodológica surgió de esta corrida y se aplica a toda la tesis: los modelos de una misma semilla en distintos escenarios son casi idénticos, por lo que una prueba que trata cada combinación de semilla y escenario como un bloque independiente (12 bloques) está pseudorreplicada y exagera la significancia. Las pruebas del Capítulo 5 usan como unidad el modelo.

Apéndice B

Eje sintético

Elliptic no publica el significado de sus variables ni anota qué patrón de lavado sigue cada transacción ilícita, de modo que sobre él solo se puede medir la estabilidad de las explicaciones, no si señalan lo correcto. Para observar esa segunda propiedad se construyó un grafo sintético de transacciones en el que cada transacción ilícita pertenece a una tipología conocida y en el que se sabe qué aristas y qué variables la delatan. Este anexo resume sus resultados como evidencia complementaria; no forma parte del contraste de hipótesis.

B.1. Diseño

El generador (`phase1/synthetic_aml_generator.py`, opción `-v4`, semilla 42) produce un grafo con tipologías de lavado insertadas sobre un fondo de transacciones legítimas. Se verificó que ninguna variable aislada permite separar las clases (AUC de una sola variable entre 0,50 y 0,62) y que las cuatro arquitecturas aprenden la tipología con un margen de al menos 0,10. Se entrenaron 144 configuraciones: las 4 arquitecturas, 4 escenarios, 3 técnicas de compensación y 3 semillas, explicadas con GNNExplainer, PGExplainer y Shapley por muestreo.

Los escenarios del eje sintético difieren de los de Elliptic: los niveles 1:50 y 1:100 se obtienen quitando transacciones ilícitas. Además se miden la **plausibilidad** (coincidencia de las variables o aristas señaladas con las que de verdad definen la tipología) y la **fidelidad** (cuánto cambia la predicción al retirar lo señalado, fid+, o al conservar solo lo señalado, fid−), cada una frente a una explicación al azar.

B.2. Resultados

La Tabla B.1 muestra el resultado central del eje: el explicador más estable no es el más correcto. Shapley por muestreo es el más estable en tres de las cuatro arquitecturas (Spearman entre

Tabla B.1: Eje sintético: estabilidad (Spearman global), plausibilidad de variables y fidelidad (fid+), con su valor al azar entre paréntesis. Medias sobre 36 configuraciones por fila. Fuente: `results_phase1_v4/summary_v4.csv`.

Arquitectura	Explicador	Estabilidad	Plausibilidad de variables	Fidelidad (fid+)
GCN	GNNExplainer	0,848	0,296 (0,150)	0,147 (0,030)
	Shapley	0,984	0,114 (0,150)	0,195 (0,063)
GraphSAGE	GNNExplainer	0,907	0,404 (0,150)	0,363 (0,048)
	Shapley	0,953	0,157 (0,150)	0,135 (0,092)
GAT	GNNExplainer	0,850	0,244 (0,150)	0,264 (0,054)
	Shapley	0,959	0,118 (0,150)	0,118 (0,101)
TAGCN	GNNExplainer	0,957	0,447 (0,150)	0,277 (0,053)
	Shapley	0,945	0,159 (0,150)	0,154 (0,083)

0,95 y 0,98), pero su plausibilidad de variables no supera al azar (entre 0,11 y 0,16 frente a 0,15). GNNExplainer es menos estable, pero es el único cuya plausibilidad de variables supera claramente al azar (entre 0,24 y 0,45), y el de mayor fidelidad en tres de las cuatro arquitecturas (en GCN lo es Shapley). La estabilidad de GNNExplainer baja con el desbalance, de 0,92 en los niveles nativo y 1:1 a 0,86 en 1:100, mientras que la de Shapley no cambia.

Este resultado apoya una de las conclusiones del estudio principal: la estabilidad es necesaria pero no suficiente, y debe complementarse con pruebas de que la explicación depende del modelo.

B.3. Advertencias

- La plausibilidad de aristas no es comparable entre arquitecturas. TAGCN ve 6 saltos y su subgrafo receptivo contiene casi todo el grafo (mediana de 9.311 nodos y 47.311 aristas), mientras que las otras tres ven 2 saltos (unos 36 nodos y 78 aristas). La plausibilidad al azar vale 0,001 en TAGCN y cerca de 0,47 en las demás, y la ganancia aparente de TAGCN se debe a esa línea base. Por eso la tabla no la incluye.
- En las arquitecturas de dos saltos, la plausibilidad de aristas de los explicadores apenas supera al azar (entre 0,01 y 0,03), y la de PGExplainer queda por debajo.
- PGExplainer falló en 24 de 432 combinaciones, todas en los escenarios 1:50 y 1:100.

Apéndice C

Reproducibilidad

C.1. Entorno

Los experimentos se ejecutaron en una estación con GPU NVIDIA RTX 4060 de 8 GB, con Python 3.12 y PyTorch Geometric. Las dependencias exactas están fijadas en `uv.lock`, en la raíz del repositorio, y se instalan con `uv sync`. El conjunto de datos Elliptic se descarga automáticamente en `data/` la primera vez.

C.2. Corrida principal (sin el cierre del mercado)

- Configuración: `configs/experiment_v4_precierre.yaml`.
- Entrenamiento y explicación: `scripts/v4/run_precierre.sh` y `scripts/v4/cola_precierre.sh`. Producen 216 modelos en `results_models_v4_precierre/` y las explicaciones en `results_v4_precierre/` y `results_v4_precierre_grad/`.
- Curvas y métricas de rendimiento: `scripts/v4/reunion_0410/curvas.py -variante precierre`.
- Medidas robustas y control de pesos aleatorios: `scripts/v4/cola_robusta_precierre.sh`, que ejecuta en secuencia `metrica_robusta.py` y `control_aleatorio.py` con `-variante precierre`.

C.3. Tablas, figuras y bibliografía del manuscrito

Las tablas y figuras se regeneran sin GPU desde los resultados guardados:

- `tesis_v4/scripts/analisis.py`: tablas LaTeX y pruebas estadísticas (`tesis_v4/tablas/`).
- `tesis_v4/scripts/figuras.py`: figuras en PDF vectorial (`tesis_v4/figuras/`).
- `tesis_v4/scripts/verificar_referencias.py`: genera `bibliografia.bib` consultando Crossref por DOI y rechaza todo lo que no sea un artículo de revista o tenga una retractación registrada. Las únicas excepciones son los artículos originales de los métodos y del conjunto de datos usados, listados en `tesis_v4/referencias/excepciones.bib` con la URL donde se verificaron.

C.4. Determinismo

Las semillas de datos (2026), de modelo (42, 43 y 44), de selección de transacciones (1234) y de los explicadores están fijadas. Aun así, el entrenamiento en GPU no es determinista a nivel de pesos, porque las agregaciones de mensajes usan sumas atómicas. Repetir la corrida reproduce las conclusiones, pero no necesariamente todos los decimales.